MICRO PROJECT REPORT

ADVANCED DATABASE SYSTEMS

SEATSURE: CONCURRENCY-SAFE CAMPUS EVENT REGISTRATION WITH AUTO-PROMOTING WAITLISTS

Submitted by:
Bhavya Sharma
[University Roll No.]

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING (AI AND FUTURE TECHNOLOGIES)

Under the guidance of
[Guide's Name]
[Designation]



Alliance School of Advanced Computing

Alliance University, Bengaluru

October-2026



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on **Advanced Database Systems** for the micro project titled "SeatSure: Concurrency-Safe Campus Event Registration with Auto-Promoting Waitlists" is submitted by Bhavya Sharma [Uni. Roll No. [University Roll No.]] in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology (Computer Science and Engineering – AI and Future Technologies) of Alliance University. It is a bonafide record of work carried out under my supervision and guidance during the academic year 2026–27. The work presented in this report is original and has not formed the basis for the award of any other degree to the candidate or to anybody else.

.....

[Guide's Name]

[Designation]



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled **SeatSure: Concurrency-Safe Campus Event Registration with Auto-Promoting Waitlists** (Advanced Database Systems), submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (AI and Future Technologies)** at **Alliance University**, is a record of project work carried out under the supervision of **[Guide's Name], [Designation]**.

The design, implementation and test results described in this report belong to the SeatSure project submitted with it. The course material, documentation and open-source libraries that the work builds on are acknowledged in the references.

This report has not been submitted, in part or in full, for any other degree or award.

Name of the Student	University Registration Number	Signature
Bhavya Sharma	[University Roll No.]	

Acknowledgement

I would like to express my sincere gratitude to everyone who helped me complete this micro project, **SeatSure: Concurrency-Safe Campus Event Registration with Auto-Promoting Waitlists**.

First, I thank my guide, **[Guide's Name]**, for their guidance, feedback and encouragement throughout the project. I am also grateful to the mentors at Kalvium, whose Advanced Database Systems livebook sessions provided the concepts on which this work is built.

I also thank my peers in my Kalvium squad for the discussions and feedback that helped shape the project.

Working on SeatSure taught me that a feature is finished only when it has been shown to work under realistic conditions, with automated tests, measurements and failure scenarios, and not merely when it works once on my own screen.

Finally, I thank my family and friends for their support and patience during the semester.

Thank you.

Bhavya Sharma

TABLE OF CONTENTS

Acknowledgement **4**

Abstract **6**

Chapter I – Introduction **7**

1.1 Problem Statement 7

1.2 Objectives 8

1.3 Scope of this Report 8

1.4 Theory 8

Chapter II – System Configuration **13**

2.1 System Architecture and Data Model 13

2.2 Algorithm 16

2.3 Source Code 17

2.4 Connection with Full Stack Web Development and Back-End Web Development 34

Chapter III – Results **36**

3.1 Testing Approach 36

3.2 Test Results Summary 36

3.3 Results 36

3.4 Discussion 41

Chapter IV – Conclusion **43**

4.1 Limitations 43

4.2 Future Scope 44

References **45**

Appendix A – Test Cases **46**

Appendix B – Concept Traceability **51**

Abstract

Student clubs at Alliance University run workshops, hackathons and talks in rooms with a fixed number of seats, yet registrations are still collected through Google Forms and WhatsApp, which cannot enforce a seat limit, keep a fair waitlist or report how events fill. This report describes the Advanced Database Systems part of **SeatSure**, an integrated micro project that replaces them with a web application. Its data layer, a PostgreSQL schema together with a MongoDB document store, guarantees that seat counts stay correct and that reads stay fast.

The relational design follows Modules 1–3 of the course: an ER model normalised to 3NF with primary, foreign, unique and CHECK constraints, including `confirmed_count <= capacity`, the no-overbooking rule enforced by PostgreSQL itself; JSONB columns for each event's custom questions and each student's answers, indexed with GIN; and partial and composite indexes matched to the application's queries. Concurrency is handled with transactions and `SELECT … FOR UPDATE` row locks, and MVCC and isolation levels are demonstrated in tests. Analytics use CTEs and window functions (`RANK`, `DENSE_RANK`, `NTILE`, `ROW_NUMBER`, running `SUM` and moving `AVG`). Prisma ORM 7 provides the schema, migrations, relation queries without N+1, interactive transactions, parameterised raw SQL and structured error mapping over a sized connection pool. MongoDB, accessed through Mongoose, stores notifications and an activity feed with validators, hooks, TTL indexes, server-side `$jsonSchema` rules and aggregation pipelines with `$facet` and `$lookup`, and a transactional outbox solves the dual-write problem between the two databases.

All 47 database tests pass. EXPLAIN ANALYZE on 50,000 events and 200,000 registrations shows the indexes speeding up the catalogue query 439×, the attendee page 476×, the JSONB answer filter 114× and a rare-category filter 3.8×. A race experiment shows unlocked transactions confirming up to 10 students for a 3-seat event, while the row lock always confirms exactly 3. A connection-pool experiment matches the queueing model and shows overload turning into fast timeouts rather than hangs.

Keywords: PostgreSQL, normalisation, integrity constraints, JSONB, GIN, partial indexes, EXPLAIN ANALYZE, MVCC, row locking, window functions, Prisma ORM, MongoDB, aggregation, transactional outbox

Chapter I – Introduction

In SeatSure the database is not only where data is stored; it is the last line of defence for the system's most important rule, that an event can never hold more confirmed students than it has seats. Application code can have bugs and requests can arrive in any order, but a well-designed database refuses to store an impossible state and serialises the decisions that must not run in parallel.

SeatSure has two kinds of user. **Students** browse upcoming events, register (or join a waitlist when an event is full), cancel, and follow their passes and notifications. **Organisers** of a club create events with their own registration questions, publish them, check attendees in, export attendee lists and read analytics about how their events fill.

The project is one application split into three folders that match the three subjects of the integrated project: `/client` (Full Stack Web Development: the React interface), `/server` (Back-End Web Development: the Express API) and `/db` (Advanced Database Systems: the PostgreSQL schema, the Prisma data-access layer and MongoDB). Each subject has its own report, and each report is written to be read on its own. This report covers **Advanced Database Systems**, whose code lives mainly in `/db`, and explains how it connects to the other two parts.

1.1 Problem Statement

Student clubs at Alliance University run workshops, hackathons and talks in venues with a fixed number of seats, but registrations are collected through Google Forms and WhatsApp. These tools cannot enforce a seat limit, so popular events get overbooked when many students register at once; waitlists are tracked by hand; seats freed by cancellations are not offered to the next person; and organisers have no data on fill rates or attendance.

SeatSure is a web application where students browse events and register with a hard seat cap that stays correct under simultaneous sign-ups, cancellations automatically promote the next waitlisted student, and organisers manage attendees and view registration analytics.

From the data side this requires a design that (1) cannot store an impossible state, such as 41 confirmed seats in a 40-seat event, even if the application has a bug; (2) serialises the decision for the last seat between concurrent transactions; (3) stores data whose shape differs from event to event (custom registration questions) without giving up indexing; (4) answers catalogue,

attendee and analytics queries quickly as the data grows; and (5) keeps a second database, MongoDB, consistent with PostgreSQL without distributed transactions.

1.2 Objectives

- Model the domain as a normalised relational schema whose constraints encode the business invariants.
- Use JSONB for genuinely variable attributes and keep them queryable with a GIN index.
- Design partial and composite indexes for the real query patterns, and prove each one with EXPLAIN ANALYZE.
- Make every multi-step change atomic and safe under concurrency with transactions and row locks.
- Produce organiser analytics with common table expressions and window functions.
- Access PostgreSQL through Prisma ORM 7 with migrations, efficient relation loading, a sized connection pool and structured error handling.
- Store notifications and activity in MongoDB with validation, automatic expiry, aggregation pipelines and index design, kept in step with PostgreSQL through a transactional outbox.

1.3 Scope of this Report

This report covers Modules 1–3 of the Advanced Database Systems course: relational database engineering with PostgreSQL (Module 1), database integration with Prisma ORM (Module 2) and document databases with MongoDB (Module 3). Lesson numbers such as **M1.14** refer to the lessons of the course livebook. Views, stored procedures and triggers are not part of Modules 1–3, so the business logic stays in the application's service layer and the analytics use CTEs and window functions instead. In-memory caching with Redis (Module 4) and vector search (Module 5) are future work.

1.4 Theory

Table 1.1 lists every database concept applied in SeatSure. The paragraphs after it explain the theory behind each group.

Table 1.1: Advanced Database concepts applied in SeatSure (Modules 1–3)

Lesson	Concept	Used in SeatSure for
M1.3–1.5	Relational foundations, ER modelling, normalisation	Seven tables in 3NF; venues and clubs as their own entities; a junction table for club organisers
M1.6	DDL and integrity constraints	Primary and foreign keys with `ON DELETE RESTRICT/CASCADE`, UNIQUE, NOT NULL, 13 CHECK constraints
M1.9	JSONB + GIN	`events.questions` and `registrations.answers`; GIN (`jsonb_path_ops`) index for `@>` filters
M1.10	Partial and composite indexes	Catalogue index on published events only; partial UNIQUE index for one active registration
M1.11	MVCC and isolation levels	READ COMMITTED for the workflows; REPEATABLE READ snapshot and error 40001 shown in tests
M1.12	Window functions	`RANK`, `DENSE_RANK`, `NTILE`, `ROW_NUMBER`, running `SUM`, moving `AVG`, `COUNT(*) OVER ()`
M1.13	Common table expressions	Analytics queries built from named steps (`event_stats`, `rated`, `calendar`, `per_club`)
M1.14	Transactions and concurrent writes	`SELECT … FOR UPDATE` on the event row inside every seat-changing transaction
M1.15	EXPLAIN ANALYZE	Before/after plans for four indexes on 50,000 events and 200,000 registrations
M2.1	Prisma 7 with the pg driver adapter	`prisma7.config.ts`, `@prisma/adapter-pg`, client generated to `db/generated/prisma`
M2.3, M2.5, M2.20	Schema, migrations, enums, deployment	`schema.prisma`, three migrations, four PostgreSQL enums, `prisma migrate deploy`
M2.8, M2.10	Prisma Client CRUD, relations, N+1	Repository functions; nested `include` with a constant number of queries
M2.11	Filtering, sorting, cursor pagination	Catalogue with `cursor`, `take: limit + 1` and a unique tiebreaker in every ordering
M2.12–2.13	Connection pooling and the pool experiment	Pool of 10 with acquire and idle timeouts; measured from 1 to 40 connections
M2.15	Interactive `$transaction`, nested writes	`withTransaction` with an isolation level, a timeout and a maximum wait
M2.19	`$queryRaw` and structured errors	Row lock, JSONB filter and analytics in parameterised SQL; `toDbError` mapping
M3.2–3.4	Documents vs rows, BSON, Mongoose schemas, validators, hooks	Notifications whose payload differs per type; a `pre('validate')` hook
M3.7, M3.12, M3.16	Aggregation pipelines, `$facet`, `$lookup`	Dashboard summary in one round trip; promotions joined to notifications
M3.8	Embedding vs referencing	Activity documents embed event and actor snapshots and reference PostgreSQL ids
M3.19	TTL indexes, `$jsonSchema`, the dual-write problem	Notifications expire after 30 days; validation inside MongoDB; transactional outbox
M3.20	Index design and `explain()`	Compound feed index proven with IXSCAN (10 documents examined, 10 returned)

1.4.1 Relational Database Engineering with PostgreSQL (Module 1)

Modelling and normalisation (M1.3–M1.5). An entity–relationship model turns a description of the domain into entities, attributes and relationships; a many-to-many relationship needs a

junction table. Normalisation removes redundancy, and with it the insert, update and delete anomalies that redundancy causes. In third normal form (3NF) every non-key attribute depends on the key, the whole key and nothing but the key.

Constraints (M1.6). Data definition language declares the rules the database itself enforces: primary keys, foreign keys with `ON DELETE` behaviour, UNIQUE, NOT NULL, defaults and CHECK constraints. Rules placed here hold for every client, including a careless script or a manual `psql` session.

JSONB and indexes (M1.9, M1.10). JSONB stores a binary JSON document in a column. It suits attributes whose shape genuinely varies, and a GIN index makes containment queries (`@>`) fast. A B-tree index speeds up lookups and ordering. A *composite* index covers several columns and must match the query's filter-then-sort pattern; a *partial* index covers only the rows that satisfy a condition (for example `status = 'PUBLISHED'`), so it is smaller and exactly matches the queries that use the same condition. A partial UNIQUE index enforces uniqueness only among those rows.

MVCC, isolation and transactions (M1.11, M1.14). PostgreSQL uses multi-version concurrency control: an update writes a new version of a row, so readers never block writers. The isolation level decides which versions a transaction sees. Under READ COMMITTED (the default) each statement sees data committed before it began; under REPEATABLE READ the whole transaction sees one snapshot, and a write that conflicts with a concurrent change fails with error 40001; SERIALIZABLE additionally prevents every anomaly that a serial order could not produce. Independently of isolation, `SELECT … FOR UPDATE` locks the selected rows, so another transaction that wants to lock them waits until the first one commits or rolls back.

Analytical SQL (M1.12, M1.13). A window function computes a value over a set of rows related to the current row (`OVER (PARTITION BY … ORDER BY …)`) without collapsing them as `GROUP BY` does. This allows rankings (`RANK`, `DENSE_RANK`), positions (`ROW_NUMBER`), buckets (`NTILE`), running totals and moving averages. Common table expressions (`WITH`) give names to intermediate results, so a long query becomes a sequence of readable steps.

Query plans (M1.15). `EXPLAIN ANALYZE` executes a query and reports the plan the optimiser chose, with the real time and row count of each node. The usual workflow is to find the most expensive node (often a sequential scan followed by a sort), add an index that serves that access pattern, and run the plan again to prove the gain.

1.4.2 Database Integration with Prisma ORM (Module 2)

Setup, schema and migrations (M2.1–M2.5, M2.20). Prisma ORM 7 connects to PostgreSQL through a driver adapter (`@prisma/adapter-pg` over the `pg` driver). The `schema.prisma` file declares models, relations, enums and indexes; each change becomes a versioned SQL migration, which development applies with `migrate dev` and production with `migrate deploy`. The generated client is fully typed.

Queries (M2.8–M2.11). The client offers typed CRUD operations. Loading relations inside a loop causes the N+1 problem, one query per row; `include` and `select` load relations in a fixed number of queries. Filtering uses compound `where` objects, sorting uses `orderBy` with a unique tiebreaker so that the order is total, and cursor pagination continues from the last row seen.

Pooling, transactions and raw SQL (M2.12–M2.19). Opening a database connection per request fails under load, so a *pool* keeps a fixed number of connections and queues requests for them; its size and timeouts matter. An interactive `$transaction(async (tx) => …)` runs several statements atomically. When the query builder cannot express something, such as `FOR UPDATE`, JSONB operators or window functions, `$queryRaw` runs parameterised SQL safely. Errors are handled centrally by their codes (P2002 unique, P2003 foreign key, P2025 not found).

1.4.3 Document Databases with MongoDB (Module 3)

Documents (M3.1–M3.4). MongoDB stores typed BSON documents in collections, each with an `_id`. Documents suit data whose shape varies by type: a waitlist notification has a position, a promotion does not, and in SQL that would mean sparse nullable columns or a table per type. Mongoose adds schemas with validators (`required`, `enum`, `min`, `maxlength`) and middleware hooks (`pre`/`post`).

Modelling and aggregation (M3.6–M3.16). The choice between *embedding* data in a document and *referencing* it by id depends on size, access pattern, update frequency and whether the child lives independently. Aggregation pipelines transform documents stage by stage (`$match`, `$group`, `$sort`, `$project`); `$facet` runs several sub-pipelines over the same input in one pass, and `$lookup` joins another collection.

Operations and consistency (M3.19, M3.20). A TTL index deletes each document automatically once its date field has passed. `$jsonSchema` validation is enforced by MongoDB itself, so it also protects against clients that bypass Mongoose. When one action must update

two databases, the *dual-write problem* appears: if the second write fails, the databases disagree. Finally, indexes are designed for the actual queries and verified with `explain()`.

Chapter II – System Configuration

The databases were installed and tested on a single laptop. Table 2.1 lists the hardware and software.

Table 2.1: Hardware and software configuration

Component	Configuration
Processor and memory	Apple M4 (10 cores), 16 GB RAM
Operating system	macOS 26.7 (arm64)
Runtime	Node.js 22.23.2, npm 10.9.8 (npm workspaces: `client`, `server`, `db`)
Relational database	PostgreSQL 18.4 (Homebrew), `psql`; databases `seatsure_dev`, `seatsure_test`, and `seatsure_bench` (created and dropped by the benchmark)
ORM	Prisma ORM 7.10.0 with `@prisma/adapter-pg` and `pg` 8.23.1; TypeScript 5.9.3 run with `tsx` 4.23.15
Document database	MongoDB 8.2.6 through Mongoose 9.11.4; `mongodb-memory-server` 11.3.0 in development and tests (MongoDB Atlas or a local `mongod` also supported)
Testing	Vitest 5.0.5 against the dedicated test database and an in-memory MongoDB
Hosted fallback (documented)	Neon or Supabase (PostgreSQL) and MongoDB Atlas free tiers

2.1 System Architecture and Data Model

Figure 2.1 shows the data layer in context. The `/db` folder is a separate package, `@seatsure/db`, that holds the schema, the migrations, the seed script, the repositories (all SQL and Prisma code) and the MongoDB models. The API imports only its repository functions and never writes SQL itself.

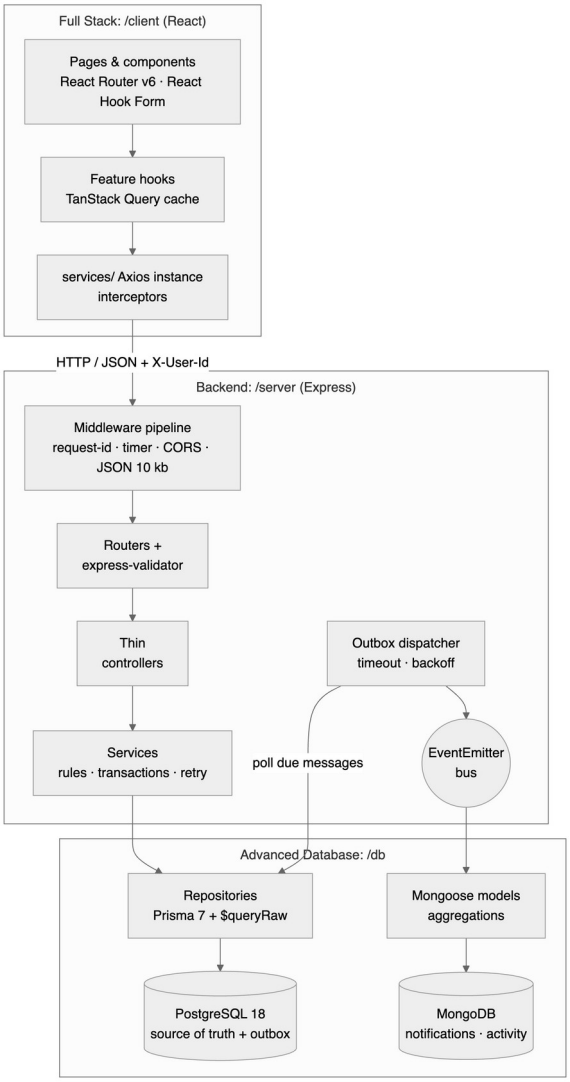


Figure 2.1: SeatSure architecture: client, API and data layers

The relational model is shown in Figure 2.2. **Users**, **clubs** and **venues** are independent entities. **Events** belong to a club, take place in a venue and are created by a user. **Registrations** link a user to an event with a status (CONFIRMED, WAITLISTED or CANCELLED). The many-to-many relationship "a user organises a club" is the junction table **club_organisers**, whose composite primary key `(club_id, user_id)` prevents duplicate memberships. **Outbox_events** stores messages for MongoDB.

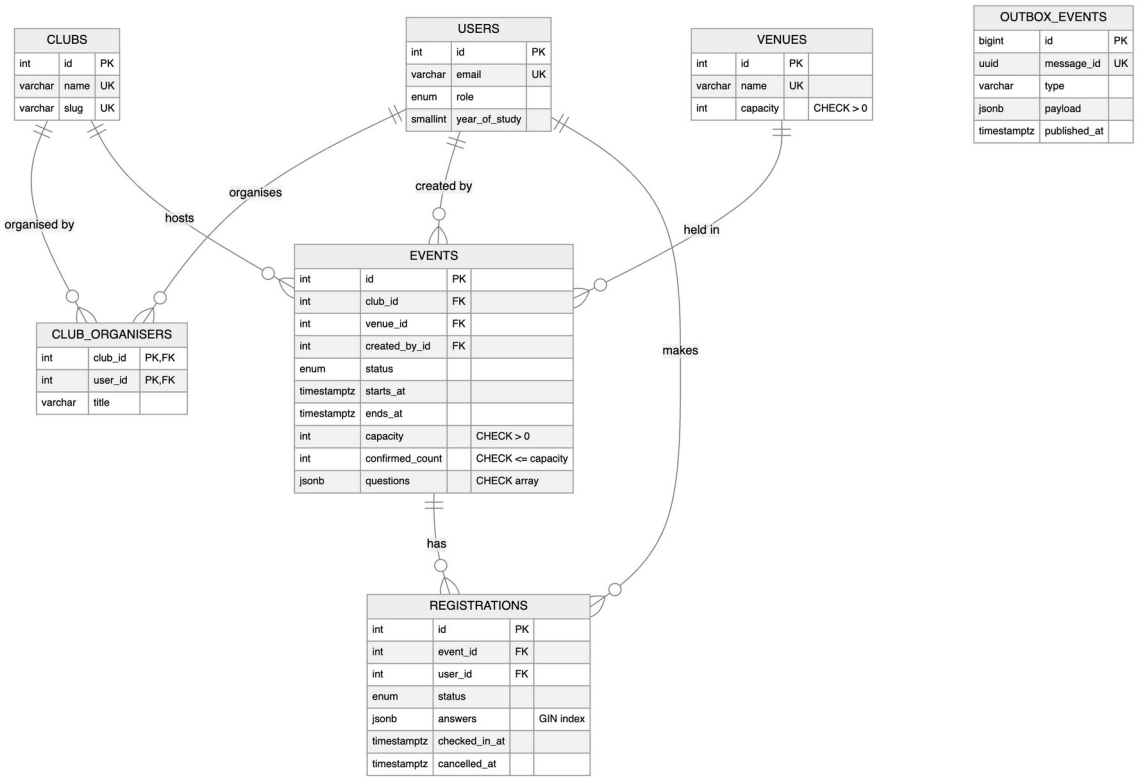


Figure 2.2: Entity–relationship diagram of the PostgreSQL schema

The schema is in third normal form. A venue's building and capacity are facts about the room, not about any one event, so they are stored once in `venues`; a club's name lives only in `clubs`. There is one deliberate, controlled exception: `events.confirmed_count` repeats the number of CONFIRMED registrations. It exists so that the seat decision reads one locked row instead of counting rows, and it is protected in three ways: it changes only inside the same transaction as the registration it counts, a CHECK constraint keeps it between 0 and the capacity, and an integrity probe (`countMismatches`, reported by the health endpoint) compares it with the real count.

Every index exists for a specific query. Table 2.2 lists them with the query each one serves and the evidence that it helps.

Table 2.2: Indexes and the queries they serve

Index	Type and definition	Query it serves	Evidence
`events_published_starts_at_idx`	Partial B-tree `(starts_at, id) WHERE status = 'PUBLISHED'`	Catalogue page; also the keyset for cursor pagination	Benchmark 439× (test DB-I01)
`events_published_category_starts_at_idx`	Partial composite `(category, starts_at, id)`	Catalogue filtered by category	Benchmark 3.8× (test DB-I02)
`events (club_id, starts_at)`	Composite B-tree	Organiser dashboard: a club's events by date	—

Index	Type and definition	Query it serves	Evidence
registrations_one_active_per_user_key	Partial UNIQUE (event_id, user_id) WHERE status <> 'CANCELLED'	At most one active registration per student and event	DB-C07, DB-C08
registrations (event_id, status, created_at)	Composite B-tree	Waitlist order; attendee page	Benchmark 476× (test DB-I04)
registrations (user_id, created_at DESC)	Composite B-tree	"My passes", newest first	—
registrations_answers_gin_idx	GIN jsonb_path_ops	Attendee filters such as answers @> '{"tshirt":"M"}'	Benchmark 114× (test DB-I03)
outbox_events_pending_idx	Partial (available_at, id) WHERE published_at IS NULL	Outbox dispatcher poll	—
MongoDB notifications	{outboxId} unique; {userId, createdAt: -1}; TTL {expiresAt}	Idempotent delivery; "my notifications"; expiry	DB-M04, DB-M05
MongoDB activity_logs	{event.id, at: -1}; {event.clubId, at: -1}; TTL {at} (180 days)	Event feed; club feed; expiry	DB-M09

2.2 Algorithm

The three central data algorithms are written below; the lesson that each step applies is given in brackets.

Algorithm 1: Allocating a seat safely

1. BEGIN a transaction at READ COMMITTED (M1.11, M2.15).
2. Lock the event row: `SELECT … FROM events e JOIN clubs c … WHERE e.id = $1 FOR UPDATE OF e`. Only the event row is locked, not the club. A concurrent transaction for the same event waits here (M1.14, M2.19).
3. Check for an active registration of the same student; the partial unique index enforces the same rule a second time (M1.10).
4. If `confirmed_count < capacity`: insert a CONFIRMED registration and run `UPDATE events SET confirmed_count = confirmed_count + 1`. The CHECK constraint rejects any update that would exceed the capacity (M1.6).
5. Otherwise insert a WAITLISTED registration; its position is `ROW_NUMBER()` over the event's waitlist ordered by `(created_at, id)` (M1.12).
6. Insert an outbox row and COMMIT. The lock is released and the next waiting transaction sees the new count.
7. On SQLSTATE 40001, 40P01 or 55P03, or a pool timeout, everything has been rolled back; retry with backoff (M2.19).

Algorithm 2: Proving an index with EXPLAIN ANALYZE

1. Load a large synthetic data set with `generate_series` (50,000 events, 200,000 registrations), then run `VACUUM ANALYZE` so that statistics and the GIN index are up to date.
2. BEGIN; `DROP INDEX` the index under test; run `EXPLAIN (ANALYZE, BUFFERS)` on the query; ROLLBACK, which brings the index back untouched (M1.15).
3. Run the same `EXPLAIN` again with the index present.
4. Compare the plan nodes (for example Seq Scan + Sort against Index Scan), the buffers read and the execution time, and report the speed-up.

Algorithm 3: Keeping MongoDB in step with PostgreSQL (transactional outbox)

1. In the same PostgreSQL transaction as the registration change, insert an `outbox_events` row; its `message_id` is a UUID generated by PostgreSQL (M3.19).
2. After commit, a dispatcher reads unpublished rows in `(available_at, id)` order through the partial index `outbox_events_pending_idx`.
3. For each row, insert the notification or activity document into MongoDB with `outboxId = message_id`. A unique index on `outboxId` turns a repeated delivery into a duplicate-key error (code 11000), which counts as success.
4. Mark the row published. On failure, increase `attempts` and move `available_at` forward (backoff).
5. To rebuild an empty MongoDB, re-queue every outbox row: PostgreSQL is the source of truth.

2.3 Source Code

This section shows how and where each concept is implemented. Every listing is copied from the project, with its file path and line numbers; lines marked “omitted” were left out for brevity. Comments carry tags such as `[DATABASE M1.14]`, so `grep -rn "DATABASE M" db` lists every place a concept is used, and Appendix B maps each concept to its files and tests.

2.3.1 Schema, keys, enums and indexes (M1.4–M1.10, M2.3, M2.5)

The schema is declared in `schema.prisma`. The event model shows foreign keys with explicit delete rules (a club or venue with events cannot be deleted), the controlled counter, the JSONB questions column and the two partial catalogue indexes (Listing 2.1).

Listing 2.1: Event model with relations and partial indexes (`db/prisma/schema.prisma`, lines 103–138)

```
103  model Event {
104    id                  Int           @id @default(autoincrement())
105    clubId              Int           @map("club_id")
106    venueId             Int           @map("venue_id")
107    createdById         Int           @map("created_by_id")
       // … 7 lines omitted …
115    capacity            Int
116    // Controlled denormalisation: kept equal to COUNT(confirmed registrations)
           inside the same
117    // transaction that changes a registration, and guarded by CHECK
           (confirmed_count <= capacity).
118    confirmedCount      Int           @default(0) @map("confirmed_count")
119    // [DATABASE M1.9: JSONB] Per-event custom questions; the shape differs per
           event.
120    questions           Json          @default("[]")
       // … 3 lines omitted …
124
125    club          Club           @relation(fields: [clubId], references: [id],
           onDelete: Restrict)
126    venue         Venue          @relation(fields: [venueId], references: [id],
           onDelete: Restrict)
127    createdBy     User           @relation("EventCreatedBy", fields:
           [createdById], references: [id], onDelete: Restrict)
128    registrations Registration[]
129
130    // [DATABASE M1.10: Composite index] Organiser dashboard: a club's events in
           date order.
131    @@index([clubId, startsAt])
132    // [DATABASE M1.10: Partial + composite indexes] The public catalogue only
           ever reads PUBLISHED
133    // events ordered by (starts_at, id) (also the keyset used for cursor
           pagination), so the
134    // index skips drafts/cancelled rows entirely. Before/after plans:
           docs/test-results/explain-analyze-*.txt.
135    @@index([startsAt, id], where: raw("status = 'PUBLISHED'"), map:
           "events_published_starts_at_idx")
136    @@index([category, startsAt, id], where: raw("status = 'PUBLISHED'"), map:
           "events_published_category_starts_at_idx")
137    @@map("events")
138  }
```

The registration model adds the partial UNIQUE index, which allows a student to cancel and register again (cancelled rows are kept as history) while forbidding two *active* registrations, and the GIN index on the JSONB answers (Listing 2.2).

Listing 2.2: Registration model: composite, partial unique and GIN indexes (`db/prisma/schema.prisma`, lines 140–165)

```
140  model Registration {
141    id          Int                @id @default(autoincrement())
142    eventId     Int                @map("event_id")
143    userId      Int                @map("user_id")
144    status      RegistrationStatus
145    // [DATABASE M1.9: JSONB] Answers to the event's custom questions, e.g.
           {"tshirt":"M","laptop":true}.
146    answers     Json               @default("{}")
       // … 5 lines omitted …
152
153    event Event @relation(fields: [eventId], references: [id], onDelete: Cascade)
154    user  User  @relation(fields: [userId], references: [id], onDelete: Cascade)
155
156    // [DATABASE M1.10: Composite indexes] Waitlist order / attendee lists, and
           "my passes".
157    @@index([eventId, status, createdAt])
158    @@index([userId, createdAt(sort: Desc)])
159    // [DATABASE M1.10 + M1.6: Partial UNIQUE index] At most one *active*
           registration per student per
160    // event, while cancelled rows are kept as history (a student may cancel and
           register again).
161    @@unique([eventId, userId], where: raw("status <> 'CANCELLED'"), map:
           "registrations_one_active_per_user_key")
162    // [DATABASE M1.9: GIN index on JSONB] Organisers filter attendees by an
           answer: answers @> '{"tshirt":"M"}'.
163    @@index([answers(ops: JsonbPathOps)], type: Gin, map:
           "registrations_answers_gin_idx")
164    @@map("registrations")
165  }
```

Prisma's schema language has no CHECK constraints, so the business invariants are added in a hand-written migration. The most important one is `events_confirmed_within_capacity_chk`: even a buggy service or a manual `UPDATE` cannot store an overbooked event (DB-C03) (Listing 2.3).

Listing 2.3: CHECK constraints (hand-written migration)
(`db/prisma/migrations/20261026092412_integrity_checks/migration.sql`, lines 15–33)

```
15  -- Events ------------------------------------------------------------------
        -------
16  ALTER TABLE "events"
17    ADD CONSTRAINT "events_capacity_positive_chk" CHECK (capacity > 0),
18    -- The core "can't overbook" invariant, enforced by PostgreSQL itself.
19    ADD CONSTRAINT "events_confirmed_within_capacity_chk" CHECK (confirmed_count
          >= 0 AND confirmed_count <= capacity),
20    ADD CONSTRAINT "events_time_order_chk" CHECK (ends_at > starts_at),
21    ADD CONSTRAINT "events_registration_closes_before_start_chk"
22      CHECK (registration_closes_at IS NULL OR registration_closes_at <=
            starts_at),
23    ADD CONSTRAINT "events_published_at_chk" CHECK (status = 'DRAFT' OR
          published_at IS NOT NULL),
24    -- [DATABASE M1.9] JSONB is flexible, but the top-level shape is still
          guaranteed.
25    ADD CONSTRAINT "events_questions_is_array_chk" CHECK (jsonb_typeof(questions)
          = 'array');
26
27  -- Registrations
        ------------------------------------------------------------------------
28  ALTER TABLE "registrations"
29    ADD CONSTRAINT "registrations_answers_is_object_chk" CHECK
          (jsonb_typeof(answers) = 'object'),
30    -- cancelled_at is set exactly when the row is CANCELLED.
31    ADD CONSTRAINT "registrations_cancelled_at_chk" CHECK ((status = 'CANCELLED')
          = (cancelled_at IS NOT NULL)),
32    -- Only a confirmed attendee can be checked in.
33    ADD CONSTRAINT "registrations_checkin_requires_confirmed_chk" CHECK
          (checked_in_at IS NULL OR status = 'CONFIRMED');
```

2.3.2 Prisma client and the connection pool (M2.1, M2.12)

One Prisma client per process owns a pool of at most ten connections. Requests queue for a free connection instead of opening new ones, and a 5-second acquire timeout turns overload into a fast error instead of a hang. The session time zone is pinned to UTC because a probe found that the adapter sends timestamps without an offset; on an IST machine, midnight UTC had been stored as 18:30 on the previous day (regression test DB-Q11) (Listing 2.4).

Listing 2.4: Driver adapter with a sized pool and UTC sessions (`db/src/client.ts`, lines 23–35)

```
23  const adapter = new PrismaPg({
24    connectionString,
25    // A small fixed pool: requests queue for a free connection instead of opening
          a new one
26    // per request (which exhausts PostgreSQL's max_connections under load).
27    max: options.poolMax ?? Number(process.env.DB_POOL_MAX ?? 10),
28    // Fail fast (instead of hanging forever) when every pooled connection is
          busy.
29    connectionTimeoutMillis: 5_000,
30    idleTimeoutMillis: 30_000,
31    // The adapter sends DateTime parameters without an offset, so PostgreSQL
          reads them in the
32    // session time zone. Pinning sessions to UTC stores instants exactly (found
          by a probe on an
33    // IST machine: 00:00Z was stored as 18:30Z the previous day). Display code
          converts to IST.
34    options: '-c timezone=UTC',
35  });
```

2.3.3 Transactions and row-level locking (M2.15, M1.11, M1.14, M2.19)

`withTransaction` wraps Prisma's interactive transaction. Any thrown error rolls back every write, and database errors are re-thrown as storage-neutral `DbError` objects (Listing 2.5).

Listing 2.5: Interactive transaction with isolation level and time limits (`db/src/transaction.ts`, lines 25–35)

```
25  export async function withTransaction<T>(work: (tx: Tx) => Promise<T>, options:
        TransactionOptions = {}): Promise<T> {
26    try {
27      return await prisma.$transaction(work, {
28        isolationLevel: Prisma.TransactionIsolationLevel[options.isolationLevel ??
              'ReadCommitted'],
29        timeout: options.timeoutMs ?? 5_000,
30        maxWait: options.maxWaitMs ?? 3_000,
31      });
32    } catch (err) {
33      throw toDbError(err) ?? err;
34    }
35  }
```

Prisma's query builder has no `FOR UPDATE`, so the lock is taken with a parameterised `$queryRaw`. `FOR UPDATE OF e` locks only the event row, even though the query joins the club to read its name (Listing 2.6).

Listing 2.6: Locking the event row (`db/src/repositories/eventRepository.ts`, lines 131–148)

```
131 export async function lockForUpdate(tx: Tx, id: number): Promise<LockedEvent |
        null> {
132   const rows = await tx.$queryRaw<LockedEvent[]>`
133     SELECT e.id,
134            e.club_id                AS "clubId",
135            c.name                   AS "clubName",
136            e.title,
137            e.status::text           AS status,
138            e.starts_at              AS "startsAt",
139            e.registration_closes_at AS "registrationClosesAt",
140            e.capacity,
141            e.confirmed_count        AS "confirmedCount",
142            e.questions
143     FROM events e
144     JOIN clubs c ON c.id = e.club_id
145     WHERE e.id = ${id}
146     FOR UPDATE OF e`;
147   return rows[0] ?? null;
148 }
```

The race experiment in Listing 2.7 shows why the lock is needed. Both functions read the current state, pause for 20 ms (as a slow request would) and then write; only the second one locks first. Chapter III shows the results.

Listing 2.7: Race experiment: read-then-write without and with the row lock (`db/scripts/race-experiment.ts`, lines 45–62)

```
45  async function naive(eventId: number, userId: number) {
46    await withTransaction(async (tx) => {
47      const confirmed = await tx.registration.count({ where: { eventId, status:
            'CONFIRMED' } });
48      const event = await tx.event.findUniqueOrThrow({ where: { id: eventId } });
49      await tx.$executeRaw`SELECT pg_sleep(0.02)`;
50      await tx.registration.create({ data: { eventId, userId, status: confirmed <
            event.capacity ? 'CONFIRMED' : 'WAITLISTED', answers: {} } });
51    });
52  }
53
54  async function locked(eventId: number, userId: number) {
55    await withTransaction(async (tx) => {
56      const event = await eventRepository.lockForUpdate(tx, eventId);
57      const hasSeat = event!.confirmedCount < event!.capacity;
58      await tx.$executeRaw`SELECT pg_sleep(0.02)`;
59      await registrationRepository.create(tx, { eventId, userId, status: hasSeat ?
            'CONFIRMED' : 'WAITLISTED', answers: {} });
60      if (hasSeat) await eventRepository.changeConfirmedCount(tx, eventId, 1);
61    });
62  }
```

Isolation levels are demonstrated with two real connections. Under REPEATABLE READ, transaction A keeps reading its snapshot after B commits a change, and A's own conflicting update fails with SQLSTATE 40001, which the error mapping marks as retryable (DB-T07) (Listing 2.8).

Listing 2.8: MVCC snapshot and serialisation failure under REPEATABLE READ (`db/tests/transactions.test.ts`, lines 124–136)

```
124 it('DB-T07 REPEATABLE READ: the snapshot stays stable, and a conflicting write
        fails with 40001', async () => {
125   const event = await createEvent(base, { capacity: 10 });
126   await a.query('BEGIN ISOLATION LEVEL REPEATABLE READ');
127   const first = (await a.query('SELECT capacity FROM events WHERE id = $1',
          [event.id])).rows[0].capacity;
128   await b.query('UPDATE events SET capacity = 25 WHERE id = $1', [event.id]);
129   const second = (await a.query('SELECT capacity FROM events WHERE id = $1',
          [event.id])).rows[0].capacity;
130   expect([first, second]).toEqual([10, 10]); // MVCC: A keeps reading its
          snapshot
131
132   const conflict = await a.query('UPDATE events SET capacity = capacity + 1
          WHERE id = $1', [event.id]).catch((e) => e);
133   await a.query('ROLLBACK');
134   expect(conflict.code).toBe('40001'); // could not serialize access due to
          concurrent update
135   expect(toDbError(conflict)?.retryable).toBe(true); // the API retries these
          with backoff
136 });
```

2.3.4 Filtering, sorting, cursor pagination and relations (M2.8–M2.11)

The catalogue query combines a compound `where` (including a column-to-column comparison for "seats available"), an ordering that always ends with the unique id, and cursor pagination that fetches one extra row to learn whether another page exists (Listing 2.9).

Listing 2.9: Filtering, sorting and cursor pagination with Prisma (`db/src/repositories/eventRepository.ts`, lines 50–74)

```
50  export async function listPublished(query: EventListQuery, client: Tx = prisma)
        {
51    const limit = query.limit ?? 12;
52    const where: Prisma.EventWhereInput = {
53      status: 'PUBLISHED',
54      startsAt: { gte: query.from ?? new Date(), ...(query.to ? { lte: query.to }
            : {}) },
55      ...(query.clubId ? { clubId: query.clubId } : {}),
56      ...(query.category ? { category: query.category } : {}),
57      ...(query.q ? { title: { contains: query.q, mode: 'insensitive' as const } }
            : {}),
58      // Column-to-column comparison through a field reference (confirmed_count <
            capacity).
59      ...(query.availability === 'open' ? { confirmedCount: { lt:
            prisma.event.fields.capacity } } : {}),
60      ...(query.availability === 'full' ? { confirmedCount: { gte:
            prisma.event.fields.capacity } } : {}),
61    };
62
63    const rows = await client.event.findMany({
64      where,
65      select: cardSelect,
66      orderBy: ORDER_BY[query.sort ?? 'soonest'],
67      take: limit + 1,
68      ...(query.cursor ? { cursor: { id: query.cursor }, skip: 1 } : {}),
69    });
70
71    const hasMore = rows.length > limit;
72    const items = hasMore ? rows.slice(0, limit) : rows;
73    return { items, nextCursor: hasMore ? items[items.length - 1]!.id : null };
74  }
```

“My passes” loads each registration with its event, club and venue through a nested `include`. Prisma resolves it with one query per relation level rather than one per row, so the number of SQL statements stays the same for 2 or 10 registrations (DB-Q08) (Listing 2.10).

Listing 2.10: Relation loading without N+1 (`db/src/repositories/registrationRepository.ts`, lines 62–83)

```
62  export async function listForUser(userId: number, client: Tx = prisma) {
63    return client.registration.findMany({
64      where: { userId },
65      orderBy: [{ createdAt: 'desc' }, { id: 'desc' }],
66      include: {
67        event: {
68          select: {
69            id: true,
              // … 7 lines omitted …
77            club: { select: { id: true, name: true } },
78            venue: { select: { id: true, name: true, building: true } },
79          },
80        },
81      },
82    });
83  }
```

2.3.5 JSONB containment and window functions in raw SQL (M1.9, M1.12, M2.19)

The organiser's attendee list needs three things the query builder cannot express in one statement: JSONB containment (`answers @> $filter`, served by the GIN index), a waitlist position per row (`ROW_NUMBER()`), and the total number of matching rows returned with the page (`COUNT(*) OVER ()`), so one round trip gives both the page and the total (Listing 2.11). Every value is a bound parameter, never concatenated into the SQL string.

Listing 2.11: Attendee page: JSONB filter, ROW_NUMBER and COUNT(*) OVER () (`db/src/repositories/registrationRepository.ts`, lines 140–168)

```
140  const rows = await client.$queryRaw<(Omit<AttendeeRow, 'waitlistPosition'> & {
         waitlistPosition: bigint | null; total: bigint })[]>`
141    WITH positioned AS (
142      SELECT r.*,
143             CASE WHEN r.status = 'WAITLISTED'
144                  THEN ROW_NUMBER() OVER (PARTITION BY r.status ORDER BY
                         r.created_at, r.id)
145             END AS waitlist_position
146      FROM registrations r
147      WHERE r.event_id = ${eventId}
148    )
149    SELECT p.id,
150           p.status::text       AS status,
151           p.answers,
              // … 7 lines omitted …
159           p.waitlist_position  AS "waitlistPosition",
160           COUNT(*) OVER ()     AS total
161    FROM positioned p
162    JOIN users u ON u.id = p.user_id
163    WHERE p.answers @> ${answersFilter}::jsonb
164      ${statusFilter}
165      ${searchFilter}
166    ORDER BY CASE p.status WHEN 'CONFIRMED' THEN 0 WHEN 'WAITLISTED' THEN 1 ELSE 2
         END,
167             p.created_at, p.id
168    LIMIT ${q.pageSize} OFFSET ${offset}`;
```

Live waitlist positions use the same idea: `ROW_NUMBER()` numbers each event's waitlist in arrival order without collapsing rows (Listing 2.12).

Listing 2.12: Waitlist positions with ROW_NUMBER() (`db/src/repositories/registrationRepository.ts`, lines 92–99)

```
 92  WITH waitlist AS (
 93    SELECT r.id,
 94           ROW_NUMBER() OVER (PARTITION BY r.event_id ORDER BY r.created_at, r.id)
                AS position
 95    FROM registrations r
 96    WHERE r.status = 'WAITLISTED'
 97      AND r.event_id IN (SELECT event_id FROM registrations WHERE id IN
           (${Prisma.join(registrationIds)}))
 98  )
 99  SELECT id, position FROM waitlist WHERE id IN
       (${Prisma.join(registrationIds)})`;
```

2.3.6 Analytics with CTEs and window functions (M1.12, M1.13)

The fill-rate report builds per-event numbers in two CTEs, then ranks every event overall (`RANK`), within its club (`PARTITION BY club_id`), and compares it with its club's average fill (`AVG … OVER (PARTITION BY club_id)`) (Listing 2.13).

Listing 2.13: Fill-rate ranking: CTEs and three window functions (`db/src/repositories/analyticsRepository.ts`, lines 36–65)

```
36  WITH event_stats AS (
37    SELECT e.id, e.title, e.club_id, c.name AS club_name, e.starts_at, e.capacity,
          e.confirmed_count,
38           COUNT(r.id) FILTER (WHERE r.status = 'WAITLISTED')       AS waitlisted,
39           COUNT(r.id) FILTER (WHERE r.checked_in_at IS NOT NULL)   AS checked_in
40    FROM events e
41    JOIN clubs c ON c.id = e.club_id
42    LEFT JOIN registrations r ON r.event_id = e.id
43    WHERE e.status = 'PUBLISHED' ${clubFilter(Prisma.sql`e.club_id`, clubIds)}
44    GROUP BY e.id, c.name
45  ),
46  rated AS (
47    SELECT *, ROUND(100.0 * confirmed_count / capacity, 1) AS fill_rate
48    FROM event_stats
49  )
50  SELECT id,
51         title,
           // … 7 lines omitted …
59         fill_rate::float8                                                    AS
               "fillRate",
60         RANK() OVER (ORDER BY fill_rate DESC, waitlisted DESC)::int          AS
               "overallRank",
61         RANK() OVER (PARTITION BY club_id ORDER BY fill_rate DESC, waitlisted
               DESC)::int AS "rankInClub",
62         ROUND(fill_rate - AVG(fill_rate) OVER (PARTITION BY club_id), 1)::float8
               AS "vsClubAverage"
63  FROM rated
64  ORDER BY "overallRank", id
65  LIMIT ${limit}`;
```

Daily sign-ups use `generate_series` to create one row per calendar day (so days without sign-ups appear as zero), a running total `SUM() OVER (ORDER BY day)` and a seven-day moving average with a `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` frame. Days are computed in the campus time zone (Listing 2.14). The club leaderboard uses `DENSE_RANK()` and `NTILE(3)` in the same way.

Listing 2.14: Daily sign-ups with a running total and a moving average (`db/src/repositories/analyticsRepository.ts`, lines 79–102)

```
79  WITH bounds AS (
80    SELECT ((now() AT TIME ZONE ${CAMPUS_TZ})::date - ${days - 1}::int) AS
          start_day,
81           (now() AT TIME ZONE ${CAMPUS_TZ})::date                      AS end_day
82  ),
83  calendar AS (
84    SELECT generate_series(start_day, end_day, interval '1 day')::date AS day FROM
          bounds
85  ),
86  daily AS (
87    SELECT (r.created_at AT TIME ZONE ${CAMPUS_TZ})::date AS day, COUNT(*) AS
          signups
88    FROM registrations r
89    JOIN events e ON e.id = r.event_id
90    WHERE r.created_at >= (SELECT start_day FROM bounds)::timestamp AT TIME ZONE
          ${CAMPUS_TZ}
91      ${eventId ? Prisma.sql`AND r.event_id = ${eventId}` : Prisma.empty}
92      ${clubFilter(Prisma.sql`e.club_id`, clubIds)}
93    GROUP BY 1
94  )
95  SELECT c.day,
96         COALESCE(d.signups, 0)::int
               AS signups,
97         SUM(COALESCE(d.signups, 0)) OVER (ORDER BY c.day)::int
               AS cumulative,
98         ROUND(AVG(COALESCE(d.signups, 0)) OVER (ORDER BY c.day
99               ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 2)::float8
                   AS "movingAvg7d"
100 FROM calendar c
101 LEFT JOIN daily d ON d.day = c.day
102 ORDER BY c.day`;
```

2.3.7 Structured error handling (M2.19)

Database failures are translated into a few storage-neutral kinds, using the PostgreSQL SQLSTATE when the driver provides one and the Prisma error code otherwise (Listing 2.15). The API maps each kind to an HTTP status and retries only `WRITE_CONFLICT` and `TIMEOUT`; it never needs to know Prisma codes.

Listing 2.15: SQLSTATE and Prisma codes mapped to error kinds (`db/src/errors.ts`, lines 36–54)

```
36  const SQLSTATE_KIND: Record<string, DbErrorKind> = {
37    '23505': 'UNIQUE_VIOLATION',
38    '23503': 'FOREIGN_KEY_VIOLATION',
39    '23514': 'CHECK_VIOLATION',
40    '40001': 'WRITE_CONFLICT', // serialization_failure
41    '40P01': 'WRITE_CONFLICT', // deadlock_detected
42    '55P03': 'WRITE_CONFLICT', // lock_not_available (NOWAIT / lock_timeout)
43    '57014': 'TIMEOUT', // query_canceled (statement_timeout)
44  };
45
46  const PRISMA_CODE_KIND: Record<string, DbErrorKind> = {
47    P2002: 'UNIQUE_VIOLATION',
48    P2003: 'FOREIGN_KEY_VIOLATION',
49    P2025: 'NOT_FOUND',
50    P2034: 'WRITE_CONFLICT', // "Transaction failed due to a write conflict or a
          deadlock"
51    P2024: 'TIMEOUT', // timed out fetching a connection from the pool
52    P2028: 'TIMEOUT', // interactive transaction expired / API error
53    P1001: 'UNAVAILABLE',
54  };
```

Listing 2.16: Classifying an error, including a pool timeout (`db/src/errors.ts`, lines 71–103)

```
 71  export function toDbError(err: unknown): DbError | null {
 72    if (err instanceof DbError) return err;
 73
 74    if (err instanceof Prisma.PrismaClientKnownRequestError) {
 75      const cause = adapterCause(err);
 76      const sqlState = cause?.originalCode ?? cause?.code;
 77      const kind = (sqlState && SQLSTATE_KIND[sqlState]) ||
           PRISMA_CODE_KIND[err.code] || 'UNKNOWN';
 78      return new DbError(kind, err.message.split('\n').pop() ?? err.message, {
 79        constraint: constraintFrom(cause),
 80        sqlState,
 81        prismaCode: err.code,
 82        cause: err,
 83      });
 84    }
 85
       // … 12 lines omitted …
 98    // node-postgres pool: every connection stayed busy for longer than
           connectionTimeoutMillis.
 99    if (err instanceof Error && /timeout exceeded when trying to
           connect/i.test(err.message)) {
100      return new DbError('TIMEOUT', 'No database connection became free in time',
           { cause: err });
101    }
102    return null;
103  }
```

2.3.8 MongoDB documents, validators, hooks and TTL (M3.2–M3.4, M3.19)

A notification's `data` payload differs per type, which is exactly the case where a document fits better than a row. The Mongoose schema validates every field, a unique index on `outboxId` makes delivery idempotent, a TTL index deletes each notification after 30 days, and a `pre('validate')` hook fills in the expiry date (Listing 2.17).

Listing 2.17: Notification schema: validators, indexes, TTL and a hook (`db/src/mongo/models/Notification.ts`, lines 10–43)

```
10  const notificationSchema = new Schema(
11    {
12      // Idempotency key: the outbox message that produced this document. A unique
            index means a
13      // retried delivery can never create a duplicate notification.
14      outboxId: { type: String, required: true },
15      // Reference (not embedding): the user lives in PostgreSQL, the source of
            truth for people.
16      userId: { type: Number, required: true, min: [1, 'userId must be a positive
            integer'] },
17      type: { type: String, required: true, enum: { values:
            [...NOTIFICATION_TYPES], message: '{VALUE} is not a notification type' }
            },
18      title: { type: String, required: true, trim: true, maxlength: 120 },
19      message: { type: String, required: true, trim: true, maxlength: 500 },
20      data: { type: Schema.Types.Mixed, default: () => ({}) },
21      readAt: { type: Date, default: null },
22      expiresAt: { type: Date, required: true },
23    },
24    {
        // … 4 lines omitted …
29    },
30  );
31
32  notificationSchema.index({ outboxId: 1 }, { unique: true });
33  // "My notifications, newest first" is the only read pattern, so index exactly
        that.
34  notificationSchema.index({ userId: 1, createdAt: -1 });
35  // [DATABASE M3.19: TTL index] MongoDB's TTL monitor deletes each document once
        `expiresAt` passes.
36  notificationSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 });
37
38  // Middleware hook: default the expiry before validation runs, so `required`
        passes.
39  notificationSchema.pre('validate', function () {
40    if (!this.expiresAt) {
41      this.expiresAt = new Date(Date.now() + NOTIFICATION_TTL_DAYS * 24 * 60 * 60
            * 1000);
42    }
43  });
```

Mongoose validators protect only writes that go through Mongoose. The same rules are therefore installed in MongoDB as a `$jsonSchema` validator when the collections are created, so even a native-driver write with an invalid type is rejected with error 121 (DB-M03) (Listing 2.18, Listing 2.19).

Listing 2.18: Server-side $jsonSchema validator (`db/src/mongo/jsonSchemas.ts`, lines 23–37)

```
23  export const notificationJsonSchema = {
24    bsonType: 'object',
25    required: ['outboxId', 'userId', 'type', 'title', 'message', 'expiresAt',
          'createdAt'],
26    properties: {
27      outboxId: { bsonType: 'string', minLength: 1 },
28      userId: { bsonType: ['int', 'long', 'double'], minimum: 1 },
29      type: { enum: [...NOTIFICATION_TYPES] },
30      title: { bsonType: 'string', minLength: 1, maxLength: 120 },
31      message: { bsonType: 'string', minLength: 1, maxLength: 500 },
32      data: { bsonType: 'object' },
33      readAt: { bsonType: ['date', 'null'] },
34      expiresAt: { bsonType: 'date' },
35      createdAt: { bsonType: 'date' },
36    },
37  } as const;
```

Listing 2.19: Creating collections with their validators and indexes (`db/src/mongo/connection.ts`, lines 57–65)

```
57  for (const spec of specs) {
58    const validator = { $jsonSchema: spec.schema };
59    if (existing.has(spec.name)) {
60      await db.command({ collMod: spec.name, validator, validationLevel: 'strict',
            validationAction: 'error' });
61    } else {
62      await db.createCollection(spec.name, { validator, validationLevel: 'strict',
            validationAction: 'error' });
63    }
64    await spec.model.syncIndexes();
65  }
```

2.3.9 Embedding versus referencing, and index design (M3.8, M3.20)

An activity document embeds small snapshots of the event and the actor. They are always read together with the activity and must show what was true at that moment, so a later title change does not rewrite history; one query renders a whole feed without joins. The ids inside the snapshots are references back to PostgreSQL for anything that needs current data. The compound indexes match the two feed queries exactly, and `explain()` confirms an index scan that examines only the 10 documents it returns (DB-M09) (Listing 2.20).

Listing 2.20: Activity log: embedded snapshots and feed indexes (`db/src/mongo/models/ActivityLog.ts`, lines 14–39)

```
14  const activitySchema = new Schema(
15    {
16      outboxId: { type: String, required: true },
17      action: { type: String, required: true, enum: [...ACTIVITY_ACTIONS] },
18      actor: new Schema({ id: { type: Number, required: true }, name: { type:
            String, required: true } }, snapshotOptions),
19      event: new Schema(
20        {
21          id: { type: Number, required: true },
22          title: { type: String, required: true },
23          clubId: { type: Number, required: true },
24          clubName: { type: String },
25        },
26        snapshotOptions,
27      ),
28      meta: { type: Schema.Types.Mixed, default: () => ({}) },
29      at: { type: Date, required: true, default: () => new Date() },
30    },
31    { collection: 'activity_logs', versionKey: false, minimize: false },
32  );
33
34  activitySchema.index({ outboxId: 1 }, { unique: true });
35  // [DATABASE M3.20: Index design] Supports the two feed queries: one event's
        feed and one club's feed.
36  activitySchema.index({ 'event.id': 1, at: -1 });
37  activitySchema.index({ 'event.clubId': 1, at: -1 });
38  // [DATABASE M3.19: TTL index] Activity older than 180 days is removed
        automatically.
39  activitySchema.index({ at: 1 }, { expireAfterSeconds: ACTIVITY_TTL_DAYS * 24 *
        60 * 60 });
```

2.3.10 Aggregation pipelines with $facet and $lookup (M3.7, M3.12, M3.16)

The organiser dashboard's activity summary is computed in a single round trip: `$match` narrows the documents to the period and the organiser's clubs first (so later stages process less data), then `$facet` runs independent sub-pipelines for totals, counts per action and the most active events (Listing 2.21).

Listing 2.21: Dashboard summary with $match and $facet (`db/src/mongo/aggregations.ts`, lines 18–54)

```
18  export function activitySummaryPipeline({ since, clubIds }: { since: Date;
        clubIds?: number[] }): PipelineStage[] {
19    return [
20      { $match: { at: { $gte: since }, ...(clubIds?.length ? { 'event.clubId': {
            $in: clubIds } } : {}) } },
21      {
22        $facet: {
23          totals: [
24            { $group: { _id: null, activities: { $sum: 1 }, events: { $addToSet:
                '$event.id' } } },
25            { $project: { _id: 0, activities: 1, uniqueEvents: { $size: '$events'
                } } },
26          ],
27          byAction: [
28            { $group: { _id: '$action', count: { $sum: 1 } } },
29            { $sort: { count: -1, _id: 1 } },
30            { $project: { _id: 0, action: '$_id', count: 1 } },
31          ],
            // … 5 lines omitted …
37          topEvents: [
38            {
39              $group: {
40                _id: '$event.id',
                  // … 9 lines omitted …
50          ],
51        },
52      },
53    ];
54  }
```

A second pipeline answers “did promoted students notice?”: it groups waitlist promotions per event and uses a pipeline `$lookup` into the notifications collection to count how many promotion notices exist and how many are still unread (Listing 2.22).

Listing 2.22: Joining collections with a pipeline $lookup (`db/src/mongo/aggregations.ts`, lines 73–100)

```
 73  export function promotionReachPipeline(clubIds?: number[]): PipelineStage[] {
 74    return [
 75      { $match: { action: 'PROMOTED', ...(clubIds?.length ? { 'event.clubId': {
           $in: clubIds } } : {}) } },
 76      { $group: { _id: '$event.id', title: { $last: '$event.title' }, promotions:
           { $sum: 1 } } },
 77      {
 78        $lookup: {
 79          from: 'notifications',
 80          let: { eventId: '$_id' },
 81          pipeline: [
 82            { $match: { type: 'PROMOTED_FROM_WAITLIST', $expr: { $eq:
                 ['$data.eventId', '$$eventId'] } } },
 83            { $project: { _id: 0, readAt: 1 } },
 84          ],
 85          as: 'notices',
 86        },
 87      },
 88      {
 89        $project: {
 90          _id: 0,
 91          eventId: '$_id',
 92          title: 1,
 93          promotions: 1,
 94          notified: { $size: '$notices' },
 95          unread: { $size: { $filter: { input: '$notices', as: 'n', cond: { $eq:
                 ['$$n.readAt', null] } } } },
 96        },
 97      },
 98      { $sort: { promotions: -1, eventId: 1 } },
 99    ];
100  }
```

2.3.11 The transactional outbox and idempotent delivery (M3.19)

Writing to PostgreSQL and then to MongoDB from the same request is a dual write: if the second write fails, the databases disagree. SeatSure writes the message into PostgreSQL instead, inside the same transaction as the change, and a dispatcher delivers it later (Listing 2.23). On the MongoDB side, a duplicate `outboxId` means the message was already delivered before a crash or retry, so it counts as success (Listing 2.24).

Listing 2.23: Outbox: written in the business transaction, read by the dispatcher (`db/src/repositories/outboxRepository.ts`, lines 11–23)

```
11  export async function add(tx: Tx, type: string, payload: Prisma.InputJsonValue)
        {
12    return tx.outboxEvent.create({ data: { type, payload }, select: { id: true }
          });
13  }
14
15  /** Oldest pending messages that are due (uses the partial index
        outbox_events_pending_idx). */
16  export async function fetchDue(limit: number, now: Date = new Date(), client: Tx
        = prisma): Promise<OutboxMessage[]> {
17    return client.outboxEvent.findMany({
18      where: { publishedAt: null, availableAt: { lte: now } },
19      orderBy: [{ availableAt: 'asc' }, { id: 'asc' }],
20      take: limit,
21      select: { id: true, messageId: true, type: true, payload: true, attempts:
            true },
22    });
23  }
```

Listing 2.24: Idempotent insert on the MongoDB side (`db/src/mongo/store.ts`, lines 15–23)

```
15  async function insertOnce<T>(create: () => Promise<T>): Promise<'inserted' |
        'duplicate'> {
16    try {
17      await create();
18      return 'inserted';
19    } catch (err) {
20      if (isDuplicateKey(err)) return 'duplicate';
21      throw err;
22    }
23  }
```

2.3.12 Measuring an index with EXPLAIN ANALYZE (M1.15)

The benchmark measures each index fairly: the “before” plan runs inside a transaction that drops the index and is then rolled back, so the index returns untouched and both plans run on identical data (Listing 2.25).

Listing 2.25: Before/after measurement inside a rolled-back transaction (`db/src/bench/explain.ts`, lines 76–87)

```
76  export async function measureScenario(client: pg.Client, scenario: Scenario):
        Promise<ScenarioResult> {
77    let before: ExplainRun;
78    await client.query('BEGIN');
79    try {
80      for (const index of scenario.indexes) await client.query(`DROP INDEX IF
            EXISTS "${index}"`);
81      before = await explain(client, scenario.sql, scenario.params);
82    } finally {
83      await client.query('ROLLBACK'); // the indexes come back untouched
84    }
85    const after = await explain(client, scenario.sql, scenario.params);
86    return { scenario, before, after, speedup: before.executionMs /
          Math.max(after.executionMs, 0.001) };
87  }
```

2.4 Connection with Full Stack Web Development and Back-End Web Development

The data layer is shaped by the queries the other two parts make. Figure 2.3 follows one registration through all three layers.

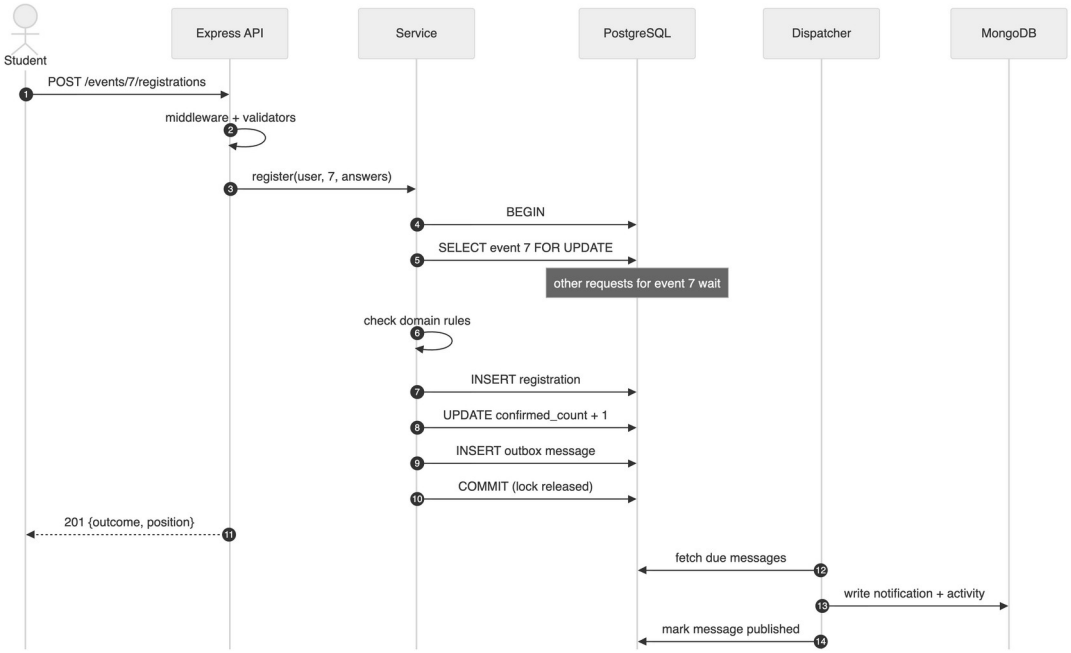


Figure 2.3: One registration across the client, the API and the two databases

- **Serving the API (Back-End).** The API calls repository functions such as `lockForUpdate`, `listPublished` and `fillRateRanking` and never writes SQL. Transactions are opened through `withTransaction`, and database failures arrive as `DbError` kinds that the API maps to HTTP status codes and uses to decide whether to retry.
- **Defence in depth (Back-End).** The API's service decides seats while holding the row lock, and the database enforces the same rule again with `CHECK (confirmed_count <= capacity)` and the partial unique index. Constraint names such as `registrations_one_active_per_user_key` become stable API error codes (`ALREADY_REGISTERED`).
- **Serving the interface (Full Stack).** The catalogue's infinite scroll uses cursor pagination over `(starts_at, id)`, the same order as the partial catalogue index. The registration form is generated from `events.questions` (JSONB), the organiser's answer filter becomes a GIN-indexed `@>` query, and the analytics page draws the window-function results and the MongoDB aggregations.

- **Two databases, one truth.** PostgreSQL holds every fact; MongoDB holds data derived from it through the outbox. If MongoDB is down, the API reports `degraded: true` for feeds while registration continues, and on restart an empty in-memory MongoDB is rebuilt by re-queuing the outbox.

Chapter III – Results

3.1 Testing Approach

The database tests run against a dedicated `seatsure_test` database, migrated with `prisma migrate deploy` before the run; every test starts from empty tables. MongoDB tests use an npm-distributed in-memory MongoDB 8.2 server whose TTL monitor runs every second, so expiry can be observed. The tests are grouped by concern: integrity constraints, transactions and isolation, query correctness, index impact and MongoDB. System-level evidence comes from three scripts: an EXPLAIN ANALYZE benchmark on a large synthetic data set, a connection-pool experiment and a race-condition experiment.

Every test case is recorded with an ID, the concept and lesson it proves, the scenario, the input, and the expected and actual results (Appendix A). Raw outputs are saved in `docs/test-results/`.

3.2 Test Results Summary

Table 3.1: Automated database tests by area (26 Oct 2026, `npm run test:db`)

Area (lessons)	Test IDs	Tests	Result
Integrity constraints: CHECK, FK, UNIQUE, ON DELETE, error mapping (M1.6, M1.9, M1.10, M2.19)	DB-C01 – C13	13	13 passed
Transactions, locking and isolation (M1.11, M1.14, M2.15)	DB-T01 – T08	8	8 passed
Query correctness: pagination, filters, window functions, CTEs, N+1, JSONB (M1.9, M1.12, M1.13, M2.10, M2.11)	DB-Q01 – Q11	11	11 passed
Index impact with EXPLAIN ANALYZE on 30,000 events and 100,000 registrations (M1.10, M1.15)	DB-I01 – I05	5	5 passed
MongoDB: validators, hooks, $jsonSchema, TTL, aggregation, explain (M3.2–M3.20)	DB-M01 – M10	10	10 passed
Total		**47**	**47 passed**

3.3 Results

- **All 47 automated tests passed** in 15.3 s on 26 October 2026.
- **Integrity (DB-C01–C12):** every constraint rejected the write it guards. For example, `UPDATE events SET confirmed_count = capacity + 1`, run directly in SQL, was rejected with SQLSTATE 23514 by `events_confirmed_within_capacity_chk`, and a second active registration was rejected by the partial unique index while re-registering after a cancellation was allowed.

- **Atomicity (DB-T01–T03):** an error after two writes, a CHECK violation in the middle of a transaction and a nested create with one invalid child each left no partial data behind.
- **Isolation and MVCC (DB-T06–T08):** READ COMMITTED read `[10, 25]` across a concurrent commit (a non-repeatable read); REPEATABLE READ read `[10, 10]` from its snapshot and its conflicting write failed with 40001; a second `FOR UPDATE` waited at least 250 ms until the first transaction committed.
- **Locking prevents overbooking (DB-T04, DB-T05):** twelve parallel read-then-write transactions on a 3-seat event oversold it without a lock, and confirmed exactly 3 (9 waitlisted, counter 3) with `FOR UPDATE`.
- **Window functions and CTEs (DB-Q04–Q07):** waitlist positions renumbered from 1, 2, 3 to 1, 2 after the first person left; fill-rate ranks, in-club ranks and differences from the club average matched hand-computed values exactly.
- **No N+1 (DB-Q08):** "My passes" issued the same number of SQL statements (at most 4) for 2 and for 10 registrations.
- **MongoDB (DB-M01–M10):** invalid documents were rejected by Mongoose and, when written with the native driver, by `$jsonSchema` (error 121); an expired notification disappeared within about a second while a fresh one remained; the same outbox message delivered twice produced one document; the feed query used an index scan that examined only the 10 documents it returned.

Index benchmark. Figure 3.1 summarises the EXPLAIN ANALYZE benchmark on 50,000 events and 200,000 registrations, and Listing 3.1 shows the plans for the catalogue query. Without the index, PostgreSQL scanned all 50,000 events, discarded 35,000 of them and sorted the rest; with the partial index it read the first 13 matching rows in order and stopped, touching 4 buffers instead of 1,064.

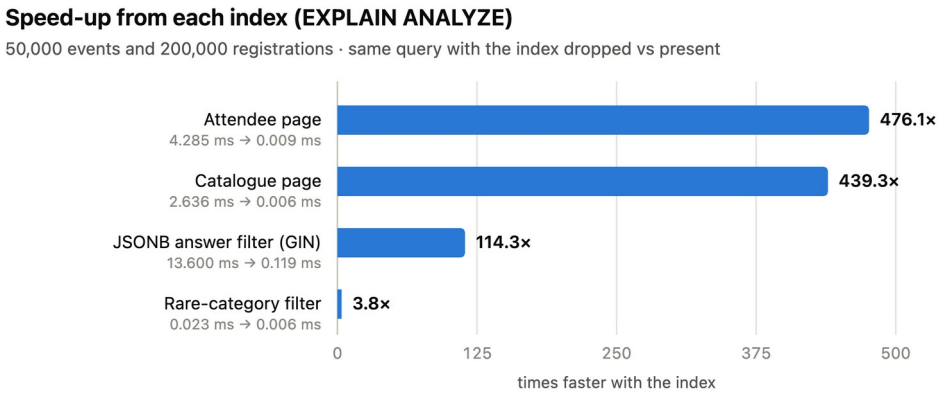


Figure 3.1: Speed-up from each index measured with EXPLAIN ANALYZE

Listing 3.1: Catalogue query plan before and after `events_published_starts_at_idx` (`docs/test-results/explain-analyze-benchmark.txt`)

```
BEFORE (index dropped): 2.636 ms
Limit (actual time=2.587..2.587 rows=13.00 loops=1)
  Buffers: shared hit=1064
  ->  Sort (actual time=2.587..2.587 rows=13.00 loops=1)
        Sort Key: starts_at, id
        Sort Method: top-N heapsort  Memory: 25kB
        Buffers: shared hit=1064
        ->  Seq Scan on events (actual time=0.817..1.882 rows=15000.00 loops=1)
              Filter: ((status = 'PUBLISHED'::"EventStatus") AND (starts_at >=
                  now()))
              Rows Removed by Filter: 35000
              Buffers: shared hit=1064
Planning Time: 0.027 ms
Execution Time: 2.590 ms
AFTER (index present):  0.006 ms
Limit (actual time=0.003..0.005 rows=13.00 loops=1)
  Buffers: shared hit=4
  ->  Index Scan using events_published_starts_at_idx on events (actual
      time=0.003..0.004 rows=13.00 loops=1)
        Index Cond: (starts_at >= now())
        Index Searches: 1
        Buffers: shared hit=4
Planning Time: 0.017 ms
Execution Time: 0.007 ms
Speed-up: 439.3x
```

Table 3.2: EXPLAIN ANALYZE benchmark (`npm run bench:explain`)

Query	Index	Before	After	Speed-up	Plan change
Catalogue page	`events_published_starts_at_idx` (partial)	2.636 ms	0.006 ms	439×	Seq Scan + Sort → Index Scan
Rare category	`events_published_category_starts_at_idx`	0.023 ms	0.006 ms	3.8×	Index + filter → composite Index Scan
JSONB answers	`registrations_answers_gin_idx` (GIN)	13.600 ms	0.119 ms	114×	Seq Scan → Bitmap Index Scan
Attendee page	`registrations_event_id_status_created_at_idx`	4.285 ms	0.009 ms	476×	Parallel Seq Scan + Sort → Index Scan

Race experiment. With N simultaneous registrations on a 3-seat event (Figure 3.2), the unlocked version confirmed every request up to 10; it stopped at 10 only because the pool runs

ten transactions at a time. The locked version confirmed exactly 3 in every run. The cost is that registrations for that one event are serialised: 40 requests took 964 ms with the lock against 124 ms without it, mostly because every transaction in the experiment includes a deliberate 20 ms pause.

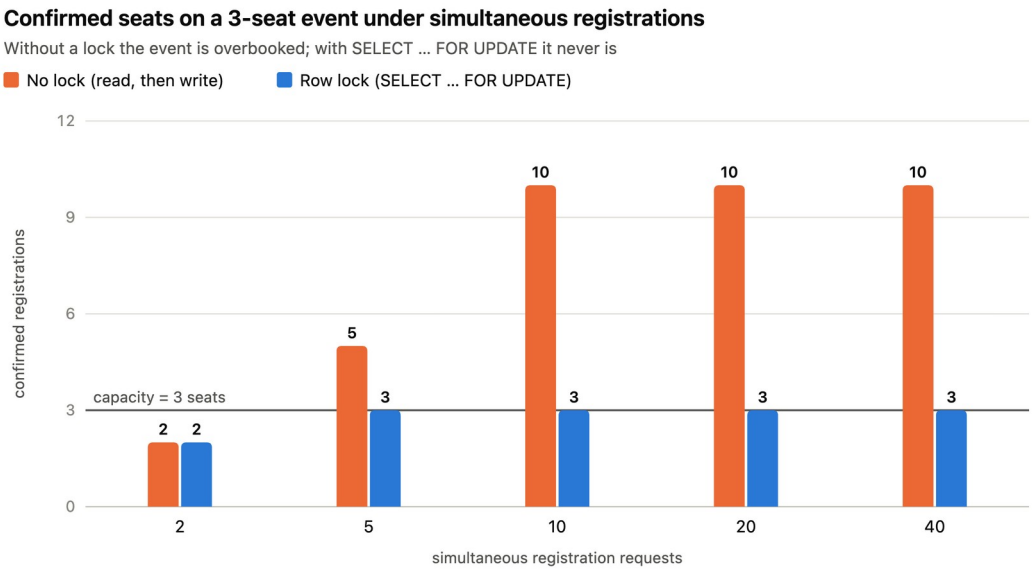


Figure 3.2: Confirmed seats on a 3-seat event under simultaneous registrations, with and without the row lock

Connection pool experiment. Forty queries that each hold a connection for 100 ms were started together with pools of different sizes (Figure 3.3). The measured times follow the queueing model ceil(40 ÷ N) × 100 ms closely: 4,106 ms with one connection, 423 ms with ten. Beyond ten connections the gain shrinks: what remains is fixed overhead, such as opening the extra connections (the experiment opens them inside the timed window) and scheduling them, and a real server would also be limited by its CPU cores and `max_connections`. With a pool of two, twenty one-second queries and a 5-second acquire timeout, ten queries succeeded and ten failed quickly with `TIMEOUT` instead of hanging (DB-C13 checks that this error is classified correctly).

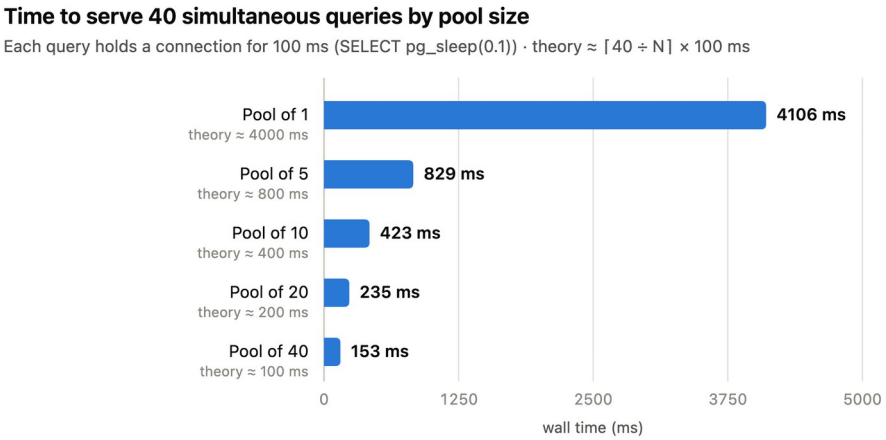


Figure 3.3: Time to serve 40 simultaneous queries by pool size

The organiser screens below show the same SQL and MongoDB results in the application.

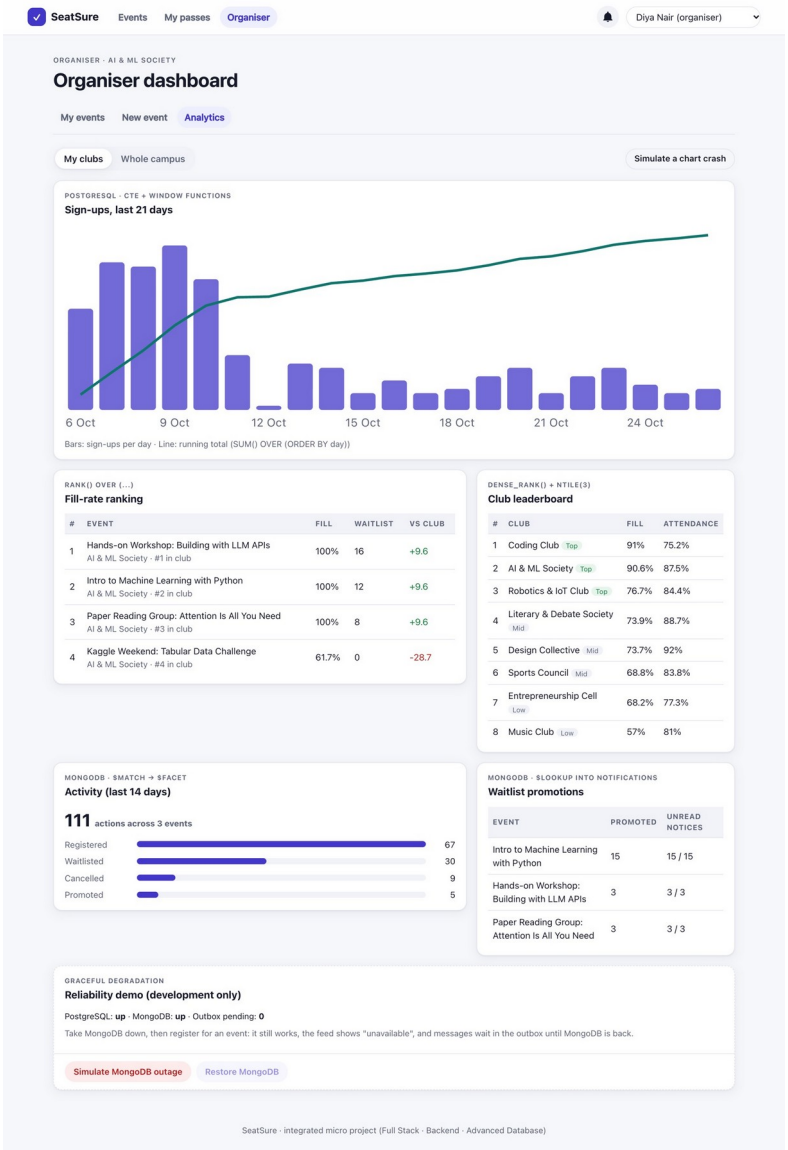


Figure 3.4: Analytics page: window-function reports from PostgreSQL and aggregation reports from MongoDB

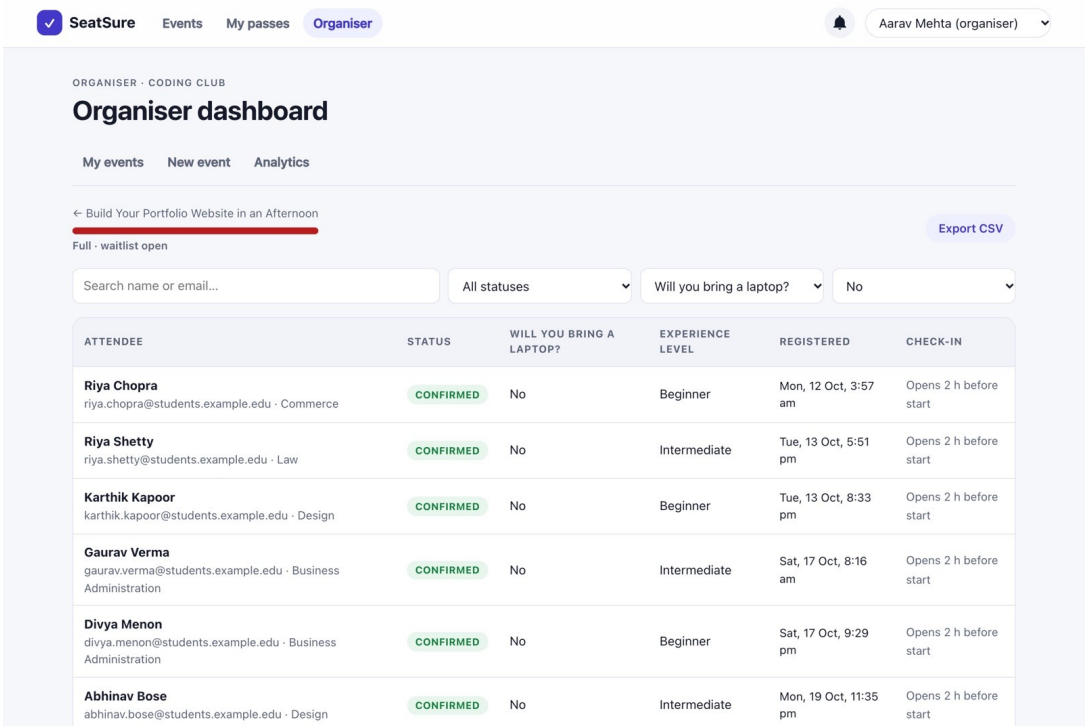


Figure 3.5: Attendee list filtered by a JSONB answer (GIN index), with waitlist positions and offset pagination

3.4 Discussion

The optimiser decides, and EXPLAIN shows its reasons. The composite category index helped only for a rare category (about 2 % of events): for common categories, the plain catalogue index plus a filter found 13 rows just as quickly, and PostgreSQL rightly ignored the composite index. The index was kept because organisers do filter by rare categories, but the report records the honest result of 3.8× rather than an inflated one (decision D-15).

Index maintenance matters. In the first benchmark run, the new GIN index was not used at all. EXPLAIN showed why: entries bulk-inserted after the index was created were still in GIN's "pending list", which must be scanned sequentially. After `VACUUM ANALYZE` merged the list, the same query read 2 index buffers instead of 493 (decision D-16).

Two real bugs were found by testing the data layer. A probe on this IST machine showed that the driver adapter sent timestamps without an offset, so PostgreSQL stored them 5½ hours off; Prisma read them back "correctly", which hid the bug from the application, but SQL using `now()` was wrong. Pinning sessions to UTC fixed it, and DB-Q11 guards against regressions (D-12). Separately, an outbox id that restarted at 1 after re-seeding collided with ids remembered by a persistent MongoDB; a UUID `message_id` generated by PostgreSQL fixed it (D-11).

Denormalisation needs a guard. The `confirmed_count` column makes the seat decision cheap, but only because it is updated in the same transaction as the registrations, bounded by a CHECK constraint and verified by an integrity probe, which reported zero mismatches after the 50-request load test.

Chapter IV – Conclusion

This report presented the Advanced Database Systems part of SeatSure. From Module 1, the PostgreSQL schema is normalised to 3NF and encodes the business invariants as constraints, including a CHECK constraint that makes overbooking impossible to store; JSONB with a GIN index handles per-event questions; partial and composite indexes were matched to the real queries and proven with EXPLAIN ANALYZE; transactions with `SELECT … FOR UPDATE` serialise the seat decision; and CTEs with window functions produce the organiser analytics. From Module 2, Prisma ORM 7 provides migrations, typed queries without N+1, cursor pagination, interactive transactions, parameterised raw SQL and structured error handling over a measured connection pool. From Module 3, MongoDB stores notifications and an activity feed with Mongoose validators and hooks, server-side `$jsonSchema` validation, TTL expiry, aggregation pipelines with `$facet` and `$lookup`, and embedding decisions backed by index design, while a transactional outbox solves the dual-write problem.

All 47 database tests pass. The benchmarks show speed-ups of 114× to 476× from the main indexes, the race experiment shows the row lock turning an overbooked event into an exact one, and the pool experiment matches the queueing model. The central lesson is that the database should enforce the rules that matter most: when the database guarantees integrity and serialises the critical decision, the application above it can be simpler and safer.

4.1 Limitations

- A single PostgreSQL server with no replication or backups beyond the developer machine.
- The per-event row lock serialises registrations for one very popular event; at much larger scale this would need a queue or a different allocation strategy.
- The attendee list uses offset pagination, which slows down on very deep pages; cursor pagination would scale better there.
- Analytics are computed on demand from live tables; with much more data they would need pre-computed summaries.
- In development MongoDB runs in memory and is rebuilt from the outbox on every restart.

4.2 Future Scope

- Cache catalogue pages and analytics in Redis (Module 4), invalidated by the outbox events.
- Pre-compute daily analytics into summary tables refreshed by a scheduled job.
- Let several dispatchers share the outbox safely with `SELECT … FOR UPDATE SKIP LOCKED`.
- Recommend events to students with vector search over event descriptions (Module 5).
- Add read replicas and point-in-time recovery for production.

References

1. Kalvium, *Advanced Database Systems (Databases for Full Stack Web Development)* livebook, Semester 3: Module 1 (Relational Database Engineering with PostgreSQL), Module 2 (Database Integration with Prisma ORM), Module 3 (Document Databases with MongoDB). Accessed 26 October 2026.
2. PostgreSQL 18 documentation: constraints; indexes (partial, multicolumn, GIN); JSON types; concurrency control (MVCC, transaction isolation, explicit locking); window functions; WITH queries; using EXPLAIN. https://www.postgresql.org/docs/18/
3. Prisma ORM documentation: driver adapters, schema reference, migrations, relation queries, pagination, transactions, raw queries, connection pool and error reference. https://www.prisma.io/docs
4. MongoDB manual: schema validation (`$jsonSchema`), TTL indexes, aggregation (`$facet`, `$lookup`), explain results. https://www.mongodb.com/docs/manual/
5. Mongoose documentation: schemas, validation and middleware. https://mongoosejs.com/docs/
6. C. Richardson, “Pattern: Transactional outbox”, microservices.io.
7. E. F. Codd, “Further Normalization of the Data Base Relational Model”, in *Data Base Systems*, Prentice-Hall, 1972.
8. SeatSure project documentation: `README.md` (traceability matrix), `DECISIONS.md`, `docs/er-diagram.md` and `docs/testing/test-cases-database.md`.

Appendix A – Test Cases

Automated tests: `npm run test:db` (Vitest), 26 October 2026, against PostgreSQL 18.4 and an in-memory MongoDB 8.2. Result: **47 / 47 passed**. Raw output: `docs/test-results/database-tests.txt`. The benchmark, pool and race tables at the end are system-level evidence and are not counted in the 47.

Table A.1: Integrity constraints (`tests/constraints.test.ts`)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
DB-C01	M1.6 CHECK constraint	Venue with no seats	`INSERT venues capacity = 0`	Rejected by `venues_capacity_positive_chk`	Rejected, SQLSTATE 23514, constraint named	Pass
DB-C02	M1.6 CHECK constraint	Event ends before it starts	`ends_at = starts_at − 1 min`	Rejected by `events_time_order_chk`	Rejected (23514)	Pass
DB-C03	M1.6 CHECK (core "no overbooking" rule)	Raw SQL bypassing the API	`UPDATE events SET confirmed_count = capacity + 1`	Rejected by `events_confirmed_within_capacity_chk`	Rejected	Pass
DB-C04	M1.9 JSONB + CHECK	Questions stored as an object, not an array	`questions = {"not":"an array"}`	Rejected by `events_questions_is_array_chk`	Rejected	Pass
DB-C05	M1.6 CHECK	CANCELLED row without `cancelled_at`	`status = CANCELLED, cancelled_at = NULL`	Rejected by `registrations_cancelled_at_chk`	Rejected	Pass
DB-C06	M1.6 CHECK	Checking in a waitlisted student	`WAITLISTED` + `checked_in_at`	Rejected by `registrations_checkin_requires_confirmed_chk`	Rejected	Pass
DB-C07	M1.10 partial UNIQUE index	Same student, two active registrations	2 inserts for the same event and user	2nd rejected: P2002 on `registrations_one_active_per_user_key`	Rejected with P2002 and that constraint	Pass
DB-C08	M1.10 partial index	Register again after cancelling	CANCELLED row, then CONFIRMED row	Allowed (cancelled rows are outside the index)	2 rows stored	Pass
DB-C09	M1.6 FOREIGN KEY	Registration for an event that doesn't exist	`event_id = 99999`	Rejected (P2003)	Rejected (FOREIGN_KEY_VIOLATION)	Pass
DB-C10	M1.6 UNIQUE	Duplicate email	Existing student's email	Rejected by `users_email_key`	Rejected	Pass

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
DB-C11	M1.6 `ON DELETE RESTRICT`	Delete a club that still has events	`DELETE club`	Refused	Refused (FK)	Pass
DB-C12	M1.6 `ON DELETE CASCADE`	Delete an event with registrations	`DELETE event`	Its registrations are removed	0 registrations left	Pass
DB-C13	M2.19 structured error handling	Pool could not supply a connection in time	`Error('timeout exceeded when trying to connect')`	Classified as `TIMEOUT` (API answers 503); unrelated errors are not database errors	As expected	Pass

Table A.2: Transactions, locking and isolation (`tests/transactions.test.ts`)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
DB-T01	M2.15 `$transaction`, M1.14 atomicity	Error thrown after two writes	Insert registration, count +1, then `throw`	Nothing persists	0 registrations, count still 0	Pass
DB-T02	M1.14 rollback	The database refuses a step mid-transaction	count +2 on a 1-seat event	CHECK violation, whole transaction rolled back	`CHECK_VIOLATION`, 0 rows	Pass
DB-T03	M2.15 nested writes	Nested create with one invalid child	Club + 2 organiser links (1 bad FK)	Nothing created	0 clubs created	Pass
DB-T04	M1.14 race condition (reproduced)	No lock: read, 20 ms gap, write	12 parallel transactions, 3 seats	More than 3 confirmed (shows the bug exists)	Oversold (> 3 confirmed)	Pass
DB-T05	M1.14 `SELECT … FOR UPDATE`	Same load, same gap, with a row lock	12 parallel transactions, 3 seats	Exactly 3 confirmed, 9 waitlisted, counter = 3	`{confirmed: 3, waitlisted: 9, stored: 3}`, 0 mismatches	Pass
DB-T06	M1.11 READ COMMITTED	Read twice while another session commits	capacity 10 → 25	Second read sees 25	`[10, 25]` (non-repeatable read)	Pass
DB-T07	M1.11 MVCC + REPEATABLE READ	Snapshot read, then a conflicting write	Same	`[10, 10]`, then the UPDATE fails with 40001	`[10, 10]`, SQLSTATE 40001, mapped as retryable	Pass
DB-T08	M1.14 row lock waiting	Second `FOR UPDATE` while the first holds the lock	Lock held 300 ms	Second waits ≥ 250 ms	Waited ≥ 250 ms	Pass

Table A.3: Query correctness (`tests/queries.test.ts`)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
DB-Q01	M2.11 cursor pagination	Walk the catalogue 3 events at a time	7 upcoming + 1 draft + 1 past	E1…E7 once each, in order, 3 pages	E1…E7, 3 pages, no draft or past event	Pass

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
DB-Q02	M2.11 filtering	Category, club, availability, title search	3 events in 2 clubs	Correct subset for each filter	All 5 filter checks match	Pass
DB-Q03	M2.11 sorting	"Most popular" sort with a tiebreak	counts 5 and 2	B before A	`[B, A]`	Pass
DB-Q04	M1.12 `ROW_NUMBER()`	Waitlist positions, then the first person leaves	3 waitlisted, cancel #1	1,2,3, then 1,2	1,2,3 → 1,2	Pass
DB-Q05	M1.12 `RANK() … PARTITION BY`, M1.13 CTE	Fill-rate ranking across 2 clubs	Fill 100 %, 50 %, 75 %	Overall 1/3/2, in-club ranks, vs club average ±25	Exact values match	Pass
DB-Q06	M1.13 CTE + `generate_series` + `SUM() OVER`	Daily sign-ups with a running total	3 sign-ups over 7 days	7 rows, last cumulative = 3, never decreasing	As expected	Pass
DB-Q07	M1.12 `DENSE_RANK`, `FILTER`	Club leaderboard with attendance	Past event, 1 of 2 checked in	`[Other Club 100 % #1]`, `[Test Club 50 %, attendance 50 % #2]`	Exact match	Pass
DB-Q08	M2.10 N+1 problem	"My passes" with event/club/venue include	2 vs 10 registrations	Same SQL count both times (≤ 4)	Constant query count	Pass
DB-Q09	M1.9 JSONB `@>`, `COUNT(*) OVER ()`	Attendee filter `tshirt = M`, page size 2	5 registrations	total 3, page 1 has 2 rows, page 2 has 1	As expected; combined filter total 3	Pass
DB-Q10	Integrity probe	Seat counter drifts from real rows	stored 0, actual 1	Mismatch reported, then `[]` after fixing	As expected	Pass
DB-Q11	Timestamps / session time zone	Prisma writes `2027-01-01T00:00Z`	Read back via SQL in UTC	`2027-01-01 00:00`, and `starts_at > now()` is true	Exact (regression test for a real bug, see DECISIONS.md D-12)	Pass

Table A.4: Index impact with EXPLAIN ANALYZE (`tests/indexes.test.ts`, 30,000 events and 100,000 registrations)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
DB-I01	M1.10 partial index, M1.15 EXPLAIN	Catalogue: next 12 published events	`WHERE status='PUBLISHED' AND starts_at >= now() ORDER BY starts_at, id LIMIT 13`	Seq Scan + Sort becomes Index Scan, faster	3.393 ms → 0.011 ms (308.5×), `Index Scan using events_published_starts_at_idx`	Pass
DB-I02	M1.10 composite index	Rare category (HACKATHON ≈ 2 %)	Same, plus `category = 'HACKATHON'`	The composite index is chosen and is faster	0.044 → 0.010 ms (4.4×), `events_published_category_starts_at_idx`	Pass

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
DB-I03	M1.9 GIN (`jsonb_path_ops`)	Who chose the Jain meal?	`answers @> '{"diet":"Jain"}'` over 100 k rows	Seq Scan becomes Bitmap Index Scan, faster	16.071 → 0.172 ms (93.4×)	Pass
DB-I04	M1.10 composite index	Attendee page in registration order	`event_id = 42 AND status = 'CONFIRMED' ORDER BY created_at LIMIT 20`	Index Scan with no separate Sort	5.260 → 0.021 ms (250.5×)	Pass
DB-I05	M1.10 partial indexes	The definitions are really partial	`pg_indexes`	All 3 index definitions contain a `WHERE` clause	3 / 3	Pass

Table A.5: MongoDB (`tests/mongo.test.ts`)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
DB-M01	M3.4 Mongoose validators	Bad notification	`type: 'SPAM'`, empty title, `userId: 0`	ValidationError on `title`, `type`, `userId`	Exactly those 3 paths	Pass
DB-M02	M3.4 middleware hook (`pre('validate')`)	Notification without an expiry	No `expiresAt`	Hook sets expiry 30 days ahead	29.9 to 30 days	Pass
DB-M03	M3.19 `$jsonSchema`	Write bypassing Mongoose (native driver)	`type: 'NOT_A_TYPE'`	MongoDB rejects it (code 121)	Code 121 DocumentValidationFailure	Pass
DB-M04	M3.19 TTL index	Expired vs fresh notification	`expiresAt` in the past	Expired one deleted automatically, fresh one kept	Deleted in about 1 s (TTL monitor), fresh kept	Pass
DB-M05	Idempotent delivery (unique `outboxId`)	Same outbox message delivered twice	Same `outboxId`	`inserted`, then `duplicate`, 1 document	As expected	Pass
DB-M06	M3.7/M3.12 aggregation, M3.16 `$facet`	Dashboard summary	8 activities (1 outside the window)	totals 7 / 2 events, per-action counts, top event	Exact match	Pass
DB-M07	M3.7 `$match` first	Scope to one club	`clubIds: [7]`	1 activity, 1 event	As expected	Pass
DB-M08	M3.16 `$lookup`	Promotions joined to notifications	2 promotions, 2 notices (1 read)	`{promotions: 2, notified: 2, unread: 1}`	Exact match	Pass
DB-M09	M3.20 index design + `explain()`	Event feed query	300 activities across 30 events	`IXSCAN` on `event.id_1_at_-1`, docs examined = docs returned	IXSCAN, 10 / 10	Pass
DB-M10	M3.8 embedding	Activity keeps its event snapshot	Old title in the snapshot	Feed reads the embedded snapshot, no join	Snapshot returned unchanged	Pass

Table A.6: EXPLAIN ANALYZE benchmark on 50,000 events and 200,000 registrations (`npm run bench:explain`)

Query	Index	Before (dropped)	After (present)	Speed-up	Plan change
Catalogue page	`events_published_starts_at_idx` (partial)	2.636 ms	0.006 ms	439×	Seq Scan + Sort → Index Scan
Rare category	`events_published_category_starts_at_idx`	0.023 ms	0.006 ms	3.8×	starts_at index + filter → composite Index Scan
JSONB answers	`registrations_answers_gin_idx` (GIN)	13.600 ms	0.119 ms	114×	Seq Scan → Bitmap Index Scan
Attendee page	`registrations_event_id_status_created_at_idx`	4.285 ms	0.009 ms	476×	Parallel Seq Scan + Sort → Index Scan

Table A.7: Connection-pool experiment (`npm run experiment:pool -w @seatsure/db`)

Pool size	Wall time	Theory ≈ ⌈40/N⌉ × 100 ms
1	4106 ms	4000 ms
5	829 ms	800 ms
10	423 ms	400 ms
20	235 ms	200 ms
40	153 ms	100 ms

Table A.8: Race-condition experiment (`npm run experiment:race -w @seatsure/db`)

Simultaneous requests	Confirmed, no lock	Confirmed, `FOR UPDATE`	Time, no lock	Time, `FOR UPDATE`
2	2	2	42 ms	54 ms
5	**5** (overbooked)	3	33 ms	119 ms
10	**10** (overbooked)	3	42 ms	246 ms
20	**10** (overbooked)	3	69 ms	489 ms
40	**10** (overbooked)	3	124 ms	964 ms

Appendix B – Concept Traceability

Every concept is tagged in the code as `// [DATABASE Mx.y: Concept]` (or `-- [DATABASE …]` in SQL). This table, taken from the project README, links each concept to the files that implement it, a way to demonstrate it, and the tests that prove it.

Table B.1: Traceability matrix: concept → code → test (Advanced Database Systems)

Module	Concept	Files / functions	How to demo	Test IDs
M1.4–1.5	ER model, normalisation (3NF)	`docs/er-diagram.md`; `prisma/schema.prisma`	Explain venue/club separation	—
M1.6	DDL constraints (PK, FK, UNIQUE, CHECK, ON DELETE)	`prisma/migrations/*_init`, `*_integrity_checks/migration.sql`	psql: `UPDATE events SET confirmed_count = capacity + 1` → rejected	DB-C01–C12
M1.9	JSONB + GIN	`events.questions`, `registrations.answers`; `registrations_answers_gin_idx`; `registrationRepository.listForEvent` (`@>`)	Attendees → filter by answer	DB-C04, DB-Q09, DB-I03
M1.10	Partial & composite indexes	`schema.prisma` `@@index(… where: …)`, partial `@@unique`	`\d events` in psql	DB-C07, DB-I01, DB-I02, DB-I04, DB-I05
M1.11	MVCC & isolation levels	`src/transaction.ts` (`withTransaction`, isolation option); `tests/transactions.test.ts`	Two psql sessions (as in tests DB-T06/T07)	DB-T06, DB-T07
M1.12	Window functions	`analyticsRepository.ts` (`RANK`, `DENSE_RANK`, `NTILE`, `SUM/AVG OVER`); `registrationRepository.waitlistPositions` (`ROW_NUMBER`), `listForEvent` (`COUNT(*) OVER()`)	Analytics page	DB-Q04–Q07, DB-Q09
M1.13	CTEs	`analyticsRepository.ts` (`event_stats`, `rated`, `calendar`, `per_club`)	—	DB-Q05–Q07
M1.14	Transactions & concurrent writes (row locks)	`eventRepository.lockForUpdate` (`FOR UPDATE OF e`); service transaction boundaries	`npm run demo:concurrency`	DB-T01–T05, DB-T08
M1.15	EXPLAIN ANALYZE	`src/bench/explain.ts`; `scripts/explain-benchmark.ts`	`npm run bench:explain`	DB-I01–I04
M2.1 / M2.12–2.13	Prisma 7 setup, adapter-pg, connection pooling	`src/client.ts` (`createPrismaClient`: `max`, timeouts, UTC session); `scripts/pool-experiment.ts`	`npm run experiment:pool -w @seatsure/db`	DB-C13, DB-Q11
M2.3 / M2.5 / M2.20	Schema, migrations, enums, `migrate deploy`	`prisma/schema.prisma`, `prisma/migrations/`, `prisma7.config.ts`	`npm run db:migrate`	(all DB tests run on migrated schema)
M2.8 / M2.10	Prisma Client CRUD, relation queries (no N+1)	`repositories/*.ts`; `registrationRepository.listForUser` (`include`)	My passes	DB-Q08

Module	Concept	Files / functions	How to demo	Test IDs
M2.11	Filtering, sorting, cursor pagination	`eventRepository.listPublished`	Catalogue	DB-Q01–Q03
M2.15	Interactive `$transaction`, nested writes	`src/transaction.ts`; `registrationService`; nested create (DB-T03)	—	DB-T01–T03
M2.19	`$queryRaw` + structured error handling	`analyticsRepository.ts`, `lockForUpdate`, `listForEvent`; `src/errors.ts` (`toDbError`)	Duplicate registration → 409	DB-C07, DB-C13, BE-U28
M3.2–3.4	Documents vs rows, BSON/_id, Mongoose schema + validators + hooks	`src/mongo/models/Notification.ts`, `ActivityLog.ts`	Notifications page	DB-M01, DB-M02
M3.7 / M3.12 / M3.16	Aggregation pipelines, `$facet`, `$lookup`	`src/mongo/aggregations.ts` (`activitySummaryPipeline`, `promotionReachPipeline`)	Analytics → Activity / Waitlist promotions	DB-M06–M08
M3.8	Embedding vs referencing	`ActivityLog.ts` (embedded snapshots + id references)	—	DB-M10
M3.19	TTL indexes, `$jsonSchema`, dual-write problem	`Notification.ts` (TTL), `jsonSchemas.ts`, `connection.ts` (`ensureCollections`); outbox (`outboxRepository.ts`)	Outage demo	DB-M03–M05, BE-I25
M3.20	Index design proven with `explain()`	`ActivityLog.ts` compound index	—	DB-M09